



DIKLAT APLIKASI ANDROID

KOMPONEN APLIKASI ANDROID

OLEH:
DWI WAHYU W

BBPPMPV BMTI
2024



DAFTAR ISI

DAFTAR ISI	
DAFTAR GAMBAR	iv
BAB I PERANGKAT ANDROID	1
TUJUAN PEMBELAJARAN:	1
A. Perkembangan Sistem Operasi Android.....	1
1. Android 1.0 & 1.1: Astro (Alpha) & Bender (Beta)	2
2. Android 1.5: Cupcake	3
3. Android 1.6: Donut	3
4. Android 2.0 & 2.1: Eclair	4
5. Android 2.2 : Froyo (Frozen Yoghurt)	5
6. Android 2.3: Gingerbread	5
7. Android 3.0 & 3.2: Honeycomb	6
8. Android 4.0: Ice Cream Sandwich.....	7
9. Android 4.1 & 4.3: Jelly Bean.....	7
10. Android 4.4: KitKat	8
11. Android 5.0 & 5.1: Lollipop.....	9
12. Android 6.0: Marshmallow	10
13. Android 7.0 & 7.1: Nougat	10
14. Android 8.0 & 8.1: Oreo	11
15. Android 9.0: Pie.....	12
16. Android 10.....	12
17. Android 11: Red Velvet Cake.....	13
B. Fitur Perangkat Keras dan Perangkat Lunak Android	14
1. Fitur Perangkat Keras Android	14
a. Touchscreen.....	14
b. GPS.....	14

c. Accelerometer	14
d. SD Card	15
2. Fitur Perangkat Lunak Android	15
a. Internet.....	15
b. Audio dan Video Support.....	15
c. Contact.....	15
d. Security	16
e. Google APIs	16
C. Kompatibilitas Perangkat dan Kompatibilitas Aplikasi Android	16
1. Pengertian “Kompatibilitas”	17
2. Memeriksa ketersediaan aplikasi ke perangkat.....	18
3. Versi Platform.....	20
minSdkVersion 15	21
targetSdkVersion 28.....	21
4. Konfigurasi layar.....	22
5. Perangkat yang Mendukung Potongan Layar	26
D. LEMBAR KERJA	28
BAB II ARSITEKTUR APLIKASI ANDROID	29
A. Arsitektur Sistem Operasi Android	29
1. Linux Kernel	30
2. Android Runtime.....	30
3. Libraries	31
4. Application Framework	31
5. Application.....	32
B. Komponen Aplikasi Android.....	33
1. Activities	33
2. Services	35

Started Services	39
IntentService	40
Bound Services	41
Mengikat ke Service	41
Life Cycle Service	42
Foreground Service.....	43
Scheduled services.....	44
3. Broadcast Receivers	44
4. Content Providers.....	47
C.Penggunaan Intent untuk Mengaktifkan Komponen dalam Aplikasi Android	48
LEMBAR KERJA	51
BAB III INTEGRATED DEVELOPMENT ENVIRONMENT ANDROID STUDIO	52
1.1 Pengenalan Android Studio	52
LEMBAR KERJA INSTALASI ANDROID STUDIO	55
1.2 Struktur Project Android Studio.....	61
BAB IV PEMBUATAN RESPONSIVE LAYOUT	66
A. Jenis dan Fungsi Layout.....	66
1. Linear Layout	66
2. Relative Layout	67
3. Frame Layout.....	70
4. Table Layout	70
5. Constraint Layout	71
B. LEMBAR KERJA.....	73
DAFTAR PUSTAKA	86

DAFTAR GAMBAR

Gambar 1 Berbagai Versi Android	1
Gambar 2 Android 1.0 & 1.1: Astro (Alpha) & Bender (Beta).....	2
Gambar 3 Android 1.5 Cupcake	3
Gambar 4 Android 1.6: Donut.....	3
Gambar 5 Android Eclair	4
Gambar 6 Android Froyo	5
Gambar 7 Android Gingerbread	5
Gambar 8 Android Honeycomb	6
Gambar 9 Android Ice Cream Sandwich.....	7
Gambar 10 Android jelly Bean.....	7
Gambar 11 Android Kitkat	8
Gambar 12 Android Lollipop	9
Gambar 13 Android Marshmallow	10
Gambar 14 Android Nougat.....	10
Gambar 15 Android Oreo	11
Gambar 16 Android Pie	12
Gambar 17 Android Q	12
Gambar 18 Android 11 Red velvet.....	13

BAB I PERANGKAT ANDROID

TUJUAN PEMBELAJARAN:

Setelah mempelajari materi ini melalui diskusi kelompok dan tanya jawab peserta mampu:

1. Menganalisis urutan perkembangan Sistem Operasi Android dengan benar
2. Membedakan fitur perangkat keras dan perangkat lunak Android dengan benar
3. Menyimpulkan kompatibilitas perangkat dengan benar
4. Menyimpulkan kompatibilitas aplikasi android dengan benar

A. Perkembangan Sistem Operasi Android

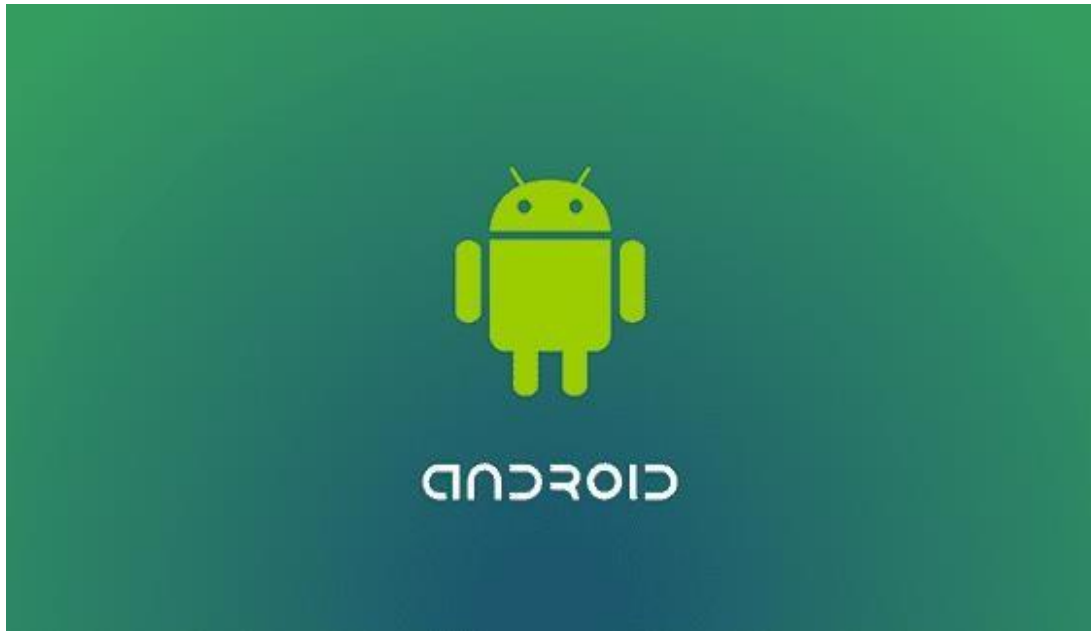
Perusahaan yang didirikan di Palo Alto, California pada Oktober 2003 ini didirikan oleh **Andy Rubin**, yang juga dikenal sebagai Bapak Android bersama **Rich Miner**, **Nick Sears**, dan **Chris White**. Pada perkembangan selanjutnya, akhirnya Android dibeli oleh Google hingga pada tahun 2008 lalu dirilis **HTC Dream**, ponsel pertama yang dibekali dengan sistem operasi Android. Urutan Perkembangan Versi Android Hingga Saat Ini



Gambar 1 Berbagai Versi Android

Berikut ini adalah penjelasan secara terperinci **urutan versi Android pertama hingga versi Android terbaru.**

1. Android 1.0 & 1.1: Astro (Alpha) & Bender (Beta)



Gambar 2 Android 1.0 & 1.1: Astro (Alpha) & Bender (Beta)

Kedua versi awal Android ini mungkin agak asing, hal ini karena versi **Android 1.0 Astro (Alpha)** dan **Android 1.1 Bender (Beta)** ini belum diluncurkan secara publik untuk kebutuhan komersil. *Platform* Android sendiri pertama kali diluncurkan pada September 2008 dengan andil Andy Rubin yang saat ini dikenal sebagai Bapak Android. Walau belum menggunakan nama makanan manis, kedua sistem operasi Android ini tentu menjadi pionir. Pasalnya, di sinilah urutan OS Android bermula lewat *smartphone* pertama, **HTC Dream**.

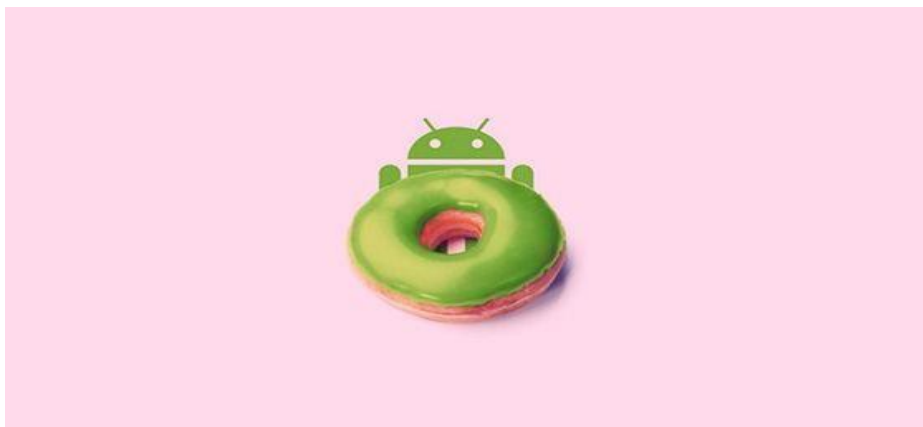
2. Android 1.5: Cupcake



Gambar 3 Android 1.5 Cupcake

Kalau versi Android pertama tadi belum menggunakan nama *dessert*, maka berbeda dengan versi Android terbaru yang rilis tahun 2009 ini. Mulai dari versi Android 1.5 ini, nama Android mulai menggunakan nama makanan manis untuk setiap versi yang diluncurkan. **Android 1.5 Cupcake** dirilis pada tanggal 30 April 2009 dengan berbagai fitur di sebuah perangkat *smartphone* untuk menggantikan *featured phone* kala itu.

3. Android 1.6: Donut



Gambar 4 Android 1.6: Donut

Tentu pada awal perilisannya, sistem operasi Android tetap memiliki banyak *bug* yang mewajibkan pengembangnya untuk mengadakan perbaikan. Hal ini pun dilakukan pada **Android 1.6 Donut** yang dirilis pada 15 September 2009. Artinya belum genap setahun semenjak perilisan Android 1.5 Cupcake atau hanya berselang 5 bulan saja. Jenis Android pun menambahkan beberapa pembaruan, terutama dukungan pada layar *smartphone* yang lebih besar.

4. Android 2.0 & 2.1: Eclair



Gambar 5 Andoid Eclair

Sama seperti versi sebelumnya, OS Android terbaru selanjutnya, yakni **Android 2.0 & 2.1 Eclair**, masih berfungsi untuk menutupi *bug* yang masih ditemukan pada sistem operasi *mobile* ini. Di samping itu, Android juga menambah berbagai fitur menarik di dalamnya yang siap digunakan oleh para pengguna *smartphone* berbasis OS Android ini. Misalnya, dukungan fitur Bluetooth hingga fitur kamera yang mulai menjadi nilai jual *smartphone* kala itu. Android 2.0 & 2.1 Eclair digunakan pada perangkat seperti **HTC Nexus One**.

5. Android 2.2 : Froyo (Frozen Yoghurt)



Gambar 6 Android Froyo

Mulai dari versi Android ini tampaknya sudah mulai dikenal luas oleh berbagai *brand smartphone*. **Android 2.2 Frozen Yoghurt** alias Froyo ini dirilis pertama kali pada tanggal 20 Mei 2010. Walaupun sudah mulai dipergunakan pada beberapa *brand*, namun tetap saja Android masih kalah bersaing dengan Symbian yang mendominasi pasar *featured phone*. Tingkatan Android 2.2 Froyo memberikan peningkatan pada kecepatan kerja, fitur USB *tethering*, WiFi *hotspot*, serta fitur keamanan.

6. Android 2.3: Gingerbread



Gambar 7 Android Gingerbread

Belum setahun berselang, **Android 2.3 Gingerbread** kembali diluncurkan pada Desember 2010 dengan berbagai peningkatan yang cukup signifikan. Hal ini terutama pada tampilan antar muka pengguna/*user interface* yang digunakan pada tipe Android yang satu ini. Mulai dari nama versi Android ini, banyak *brand smartphone* yang mulai melirik menggunakan sistem operasi Android. Salah satunya Samsung Galaxy series yang populer hingga saat ini.

7. Android 3.0 & 3.2: Honeycomb



Gambar 8 Android Honeycomb

Untuk para pengguna *smartphone* mungkin akan agak asing dengan versi Android yang satu ini. Sebab, penggunaan **Android 3.0 & 3.2 Honeycomb** yang menggunakan ikon lebah ini memang diperuntukkan khusus untuk perangkat tablet. Tentu perilsan Android 3.0 & 3.2 Honeycomb pada 10 Mei 2011 ini untuk mendukung Samsung yang mulai merilis perangkat tablet **Samsung Galaxy Tab Series** untuk menyaingi Apple iPad.

8. Android 4.0: Ice Cream Sandwich



Gambar 9 Android Ice Cream Sandwich

Selain itu, Android juga merilis versi **Android 4.0 Ice Cream Sandwich** yang kembali diperuntukkan untuk perangkat *smartphone*. Sistem Android terbaru selanjutnya ini dirilis pada 19 Oktober 2011 silam dan masih tetap menggunakan nama *dessert*, merupakan versi nama paling panjang hingga saat ini. Android 4.0 Ice Cream Sandwich memberikan banyak pembaruan, mulai dari animasi yang semakin halus, sederhana, dan mudah digunakan.

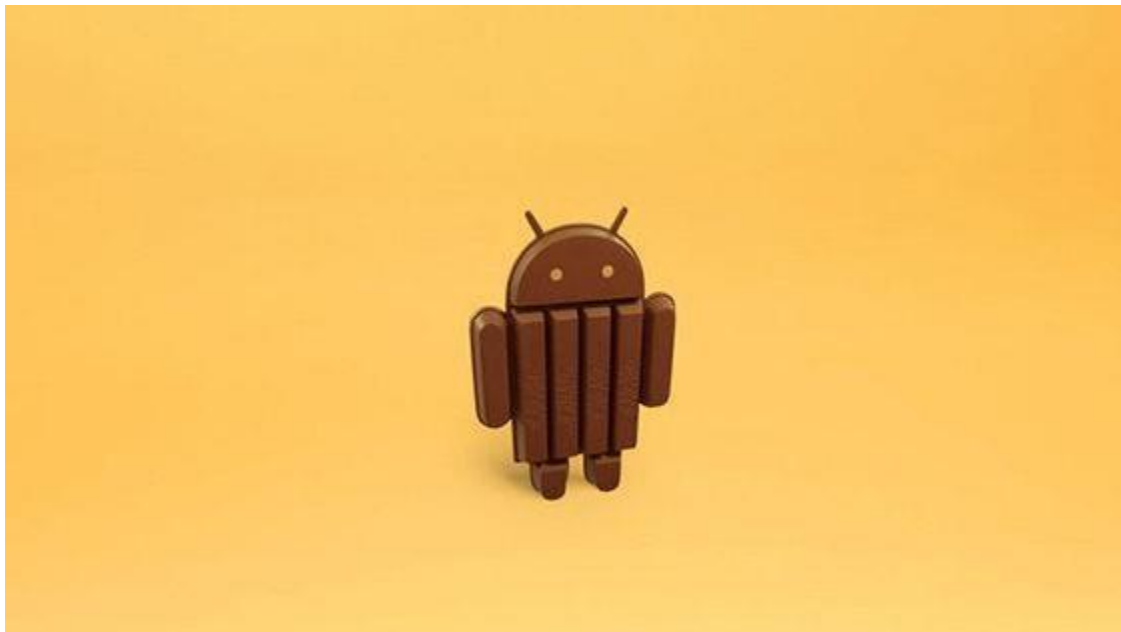
9. Android 4.1 & 4.3: Jelly Bean



Gambar 10 Android jelly Bean

Bagi para pengguna smartphone Android dari awal Android muncul, pasti menyadari peningkatan signifikan yang begitu terasa saat menggunakan **Android 4.1 & 4.3 Jelly Bean**. Sistem operasi Android ini pertama kali dirilis pada Juni 2012 dengan membawa sejumlah peningkatan terutama di sektor pengolahan grafis. Dengan begini, tentu Android 4.1 & 4.3 Jelly Bean dengan kode API level 18 bisa memberikan peningkatan fungsi pada *user interface* dan **teknologi Vsync** yang digunakannya.

10. **Android 4.4: KitKat**



Gambar 11 Android Kitkat

Menggunakan nama *brand* cemilan terkenal, **Android 4.4 KitKat** pertama kali dirilis pada Oktober 2013 lalu. Versi Android terbaik pada masanya ini pun bisa dikatakan menjadi jenis Android terfavorit karena mendukung hampir seluruh perangkat *smartphone* di dunia. Hal ini dikarenakan Android 4.4 KitKat dapat memberikan optimalisasi yang baik, termasuk pada perangkat *smartphone* yang memiliki memadai atau cukup rendah saat itu.

11. Android 5.0 & 5.1: Lollipop



Gambar 12 Android Lollipop

Mulai beberapa versi Android ke belakang, Android dan Google pun mulai secara rutin memperbarui sistem operasi mereka dalam selang waktu setahun.

Termasuk **Android**

5.0 & 5.1 Lollipop dengan API level 21 yang dirilis dan diresmikan pada Juni 2014. Bisa dibilang, seri Android ini menjadi pionir dibuatnya *smartphone flagship* dengan spesifikasi cukup mumpuni. Versi Android ini sudah mendukung arsitektur 64-bit yang sudah memungkinkan penggunaan RAM di atas 3GB. Salah satunya **Asus Zenfone 2** yang sudah mengusung RAM 4GB saat itu.

12. Android 6.0: Marshmallow



Gambar 13 Android Marshmallow

Android 6.0 Marshmallow dengan API level 23 menjadi suksesor dari versi Android sebelumnya. Sistem operasi ini pertama kali diperkenalkan pada Mei 2015 dan mulai dirilis pada Oktober 2015 silam. Nama OS Android ini secara jelas memberikan peningkatan pada sistem keamanan dengan diadakannya ***fingerprint sensor*** sebagai sistem keamanan biometrik yang digunakan. Selain digunakan untuk mengunci layar, *fingerprint sensor* ini dapat digunakan untuk otentikasi Google Play Store dan pembelian dengan menggunakan Android Pay.

13. Android 7.0 & 7.1: Nougat



Gambar 14 Android Nougat

Untuk saat ini, sistem operasi **Android 7.0 & 7.1 Nougat** mungkin masih digunakan pada beberapa *smartphone* jadul. Android 7.0 & 7.1 Nougat pertama kali diperkenalkan pada Juni 2016 dengan menampilkan ikon robot Android dengan batangan Nougat. Versi Android ini mengalami perubahan dari segi tampilan antarmuka. Selain itu, ada juga fitur ***splitscreen*** untuk membagi tampilan layar untuk dua aplikasi sekaligus.

14. Android 8.0 & 8.1: Oreo



Gambar 15 Android Oreo

Selanjutnya, **Android 8.0 & 8.1 Oreo** menjadi sistem operasi Android yang banyak digunakan pada beberapa *smartphone*, salah satunya mungkin *smartphone* yang kamu gunakan saat ini. Versi Android ini dirilis secara stabil mulai Agustus 2017 dan sudah mengalami pembaruan lewat versi Android 8.1.0 Oreo. Android 8.0 & 8.1 Oreo menjadi versi Android kedua yang menggunakan makanan manis dari nama *brand* terkenal setelah Android 4.4 KitKat. Sistem operasi Android Oreo menawarkan pengalaman *multitasking* yang makin mumpuni dibanding versi sebelumnya. Ada juga **Project Treble** yang memungkinkan pengguna mendapat pembaruan lebih cepat.

15. Android 9.0: Pie



Gambar 16 Android Pie

Kemudian ada generasi **Android 9.0 Pie** yang secara resmi diperkenalkan pada Agustus 2018. Sistem operasi Android ini memberi banyak ubahan, terutama untuk HP dengan desain baru. Misal Android 9.0 Pie memberikan navigasi berupa *gesture* yang menggantikan tombol fisik Home, Back, dan Recent Apps.

Fitur lainnya yang cukup berguna adalah sistem notifikasi, pengatur kecerahan, hingga sistem *screenshot* terbaru yang lebih memudahkan pengguna.

16. Android 10



Gambar 17 Android Q

Lalu, ada versi Android terbaru Q atau versi Android 10 yang baru diluncurkan pada 13 Maret 2019 dan saat ini masih terbatas pada beberapa perangkat HP Android saja. Versi Android tertinggi sekarang yang sudah rilis ini pertama disematkan pada seri *smartphone* Google, yakni Google Pixel, Google Pixel XL, Google Pixel 2, Google Pixel 2 XL, dan lainnya. Salah satu fitur dalam versi Android terbaru 2019 ini adalah **Dark Mode** alias mode gelap yang diklaim mampu meningkatkan performa baterai.

17. Android 11: Red Velvet Cake



Gambar 18 Android 11 Red velvet

Siklus penamaan menggunakan nama *dessert* sempat berhenti setelah Google merilis Android 10. Nah, pada versi Android terbaru 2020 yang disebut Android 11, Google kabarnya akan kembalinya dengan nama makanan manis, yakni **Red Velvet Cake**. Namun, versi Android terbaru ini

masih belum dirilis secara resmi sehingga nama Android 11 masih dipertimbangkan dan hanya digunakan oleh tim *developer* saja. Berdasarkan kabar yang beredar, ada sejumlah fitur Android 11 yang kemungkinan bisa dinikmati setelah versi ini dirilis, misalnya fitur keamanan yang lebih canggih dari sebelumnya.

B. Fitur Perangkat Keras dan Perangkat Lunak Android

1. Fitur Perangkat Keras Android

Perangkat Android memiliki beberapa fitur perangkat keras didalamnya, yang dapat dimanfaatkan developer dalam membangun aplikasi.

a. Touchscreen

Perangkat Android memiliki fitur layar sentuh yang memberikan beberapa kemungkinan bagi pengguna untuk berinteraksi dengan aplikasi dengan menggunakan jari. Pengguna dapat melakukan swipe, flip, drag, dan pinch untuk zoom. Android juga mendukung multitouch yang berarti keseluruhan layar dapat disentuh dengan satu atau lebih jari pada saat yang bersamaan.

b. GPS

Sistem operasi Android mendukung GPS yang memungkinkan developer untuk mengakses lokasi pengguna. Contoh aplikasi yang memanfaatkan GPS adalah aplikasi peta (Map) yang menunjukkan lokasi pengguna dan memberikan petunjuk untuk menuju suatu lokasi.

c. Accelerometer

Android mendukung Accelerometer yaitu perangkat yang digunakan untuk mengukur percepatan. Accelerometer dapat memberitahukan apabila suatu perangkat android bergerak atau terguncang atau berbalik arah posisinya.

d. SD Card

Android memiliki fitur yang memungkinkan pengguna atau aplikasi untuk mengakses (menyimpan atau membuka) file pada SD Card. SD Card merupakan media penyimpanan medium yang digunakan perangkat android dan beberapa perangkat mobile lain non android sebagai media penyimpanan.

2. Fitur Perangkat Lunak Android

Android memiliki banyak fitur perangkat lunak yang dapat digunakan oleh developer dalam mengembangkan aplikasi. Beberapa fitur populer yang akan dijabarkan disini.

a. Internet

Kemampuan akses internet pada Android memberikan banyak keunggulan. Berbagai informasi secara real-time dapat diperoleh dengan mudah dengan internet. Contoh, sebagai pengguna dapat menggunakan internet untuk melihat jadwal pemutaran film bioskop, cuaca suatu area, jadwal penerbangan dan lainnya. Sebagai developer dapat menggunakan internet untuk akses secara real-time kepada data, input data. Developer juga dapat menggunakan internet untuk penyimpanan berbagai asset untuk kemudian digunakan suatu aplikasi, seperti yang dilakukan Pandora dan Youtube. Dengan internet dapat dibangun model aplikasi yang disebut client-server computing. Contoh lain aplikasi peta mengakses data peta dan GPS dari web server.

b. Audio dan Video Support

Sistem operasi Android memungkinkan developer menyertakan audio dan video dalam aplikasi dengan mudah. Berbagai format audio dan video didukung.

c. Contact

Android memungkinkan akses ke contacts yang tersimpan di perangkat Android. Developer dapat menggunakan fitur ini menampilkan contacts dalam cara baru yang berbeda. Hal ini yang dapat dilakukan adalah membangun aplikasi yang

menggabungkan antara contacts dengan GPS yang memberikan notifikasi kepada pengguna jika pengguna berada didekat alamat satu contact yang ada.

d. Security

Android memungkinkan aplikasi untuk melakukan banyak hal. Akan tetapi Android juga menyiapkan mekanisme keamanan berupa permission berkaitan dengan beberapa tugas. Contoh: Download image dan penyimpanan di SD Card, maka harus disetujui terlebih dahulu permission untuk mengakses SD Card.

e. Google APIs

Sistem operasi Android memungkinkan dengan tidak terbatas membuat panggilan telepon mengorganisasi contacts atau menginstall aplikasi. Developer dapat mengintegrasikan peta kedalam suatu aplikasi dengan menggunakan Maps API yang mengandung Map Widgets. Berbagai fitur dapat ditambahkan dengan Maps API, antara lain:

- (1) Menampilkan suatu lokasi di peta
- (2) Mendapatkan panduan navigasi
- (3) Komunikasi data antara aplikasi dengan clouds

C. Kompatibilitas Perangkat dan Kompatibilitas Aplikasi Android

Android didesain untuk berjalan di banyak jenis perangkat, mulai dari ponsel hingga tablet dan TV. Sebagai developer, berbagai perangkat memberikan potensi penggunaan yang sangat besar untuk aplikasi . Agar berhasil di semua perangkat ini, aplikasi harus memberi toleransi beberapa variasi fitur dan menyediakan antarmuka pengguna fleksibel yang dapat beradaptasi dengan beragam konfigurasi layar.

Untuk memfasilitasi upaya menuju sasaran tersebut, Android menyediakan framework aplikasi dinamis tempat dapat menyediakan resource aplikasi khusus konfigurasi dalam file statis (seperti tata letak XML yang berbeda untuk ukuran layar yang berbeda). Kemudian Android akan memuat resource yang sesuai berdasarkan konfigurasi perangkat saat ini. Jadi, dengan beberapa desain aplikasi dan beberapa resource aplikasi tambahan, dapat memublikasikan paket aplikasi tunggal (APK) yang memberikan pengalaman pengguna yang dioptimalkan di berbagai perangkat. Akan tetapi, jika perlu, dapat menentukan persyaratan fitur aplikasi dan mengontrol jenis perangkat yang dapat menginstal aplikasi dari Google Play Store. Halaman ini menjelaskan cara dapat mengontrol perangkat yang dapat mengakses aplikasi, dan cara menyiapkan aplikasi untuk memastikan aplikasi tersebut menjangkau audiens yang tepat.

1. Pengertian “Kompatibilitas”

Saat membaca lebih lanjut tentang pengembangan Android, mungkin akan menemukan istilah “kompatibilitas” dalam berbagai situasi. Ada dua jenis kompatibilitas: *kompatibilitas perangkat* dan *kompatibilitas aplikasi*. Android adalah software open source, setiap produsen hardware dapat membuat perangkat yang menjalankan sistem operasi Android. Sebuah **perangkat dinyatakan "kompatibel dengan Android"** hanya jika perangkat tersebut menjalankan dengan benar aplikasi yang tertulis untuk *lingkungan eksekusi Android*. Detail persis tentang lingkungan eksekusi Android ditentukan oleh program kompatibilitas Android dan setiap perangkat harus lulus Compatibility Test Suite (CTS) agar dianggap kompatibel. Sebagai developer aplikasi, tidak perlu khawatir apakah perangkat kompatibel dengan Android, karena hanya perangkat yang kompatibel dengan Android yang menyertakan Google Play Store. Ketika mendownload aplikasi di Play Store perlu mempertimbangkan apakah **aplikasi kompatibel** dengan setiap konfigurasi perangkat yang potensial. Karena Android berjalan di berbagai konfigurasi perangkat,

beberapa fitur tidak tersedia di semua perangkat. Misalnya, beberapa perangkat mungkin tidak menyertakan sensor kompas. Jika fungsi inti aplikasi memerlukan penggunaan sensor kompas, maka aplikasi hanya kompatibel dengan perangkat yang menyertakan sensor kompas.

2. Memeriksa ketersediaan aplikasi ke perangkat

Android mendukung berbagai fitur yang dapat dimanfaatkan aplikasi melalui API platform. Beberapa fitur berbasis hardware (seperti sensor kompas), beberapa berbasis software (seperti widget aplikasi), dan beberapa bergantung pada versi platform. Tidak semua perangkat mendukung setiap fitur, jadi mungkin perlu mengontrol ketersediaan aplikasi untuk perangkat berdasarkan fitur yang diperlukan aplikasi. Untuk mencapai basis pengguna sebesar mungkin bagi aplikasi, harus terus mendukung sebanyak mungkin konfigurasi perangkat dengan APK tunggal. Di sebagian besar situasi, dapat melakukannya dengan menonaktifkan fitur opsional pada waktu proses dan menyediakan resource aplikasi dengan alternatif untuk berbagai konfigurasi (seperti beragam tata letak untuk berbagai ukuran layar). Akan tetapi, jika perlu, dapat membatasi ketersediaan aplikasi untuk perangkat melalui Google Play Store berdasarkan karakteristik perangkat berikut:

- Fitur perangkat
- Versi platform
- Konfigurasi layar

Pengelolaan ketersediaan aplikasi berdasarkan fitur perangkat, Android menentukan *ID fitur* untuk setiap fitur hardware atau software yang mungkin tidak tersedia di semua perangkat. Misalnya, ID fitur untuk sensor kompas adalah FEATURE_SENSOR_COMPASS dan ID fitur untuk widget aplikasi adalah FEATURE_APP_WIDGETS. Jika perlu, dapat mencegah pengguna menginstal aplikasi saat perangkatnya tidak menyediakan fitur tertentu dengan mendeklarasikannya dengan elemen `<uses-feature>` di file manifest aplikasi. Misalnya, jika aplikasi tidak berfungsi di perangkat yang tidak memiliki sensor kompas, dapat mendeklarasikan sensor kompas seperti yang

diwajibkan dengan tag manifest berikut:

```
<manifest ... >
    <uses-feature
        android:name="android.hardware.sensor.compass"
        android:required="true" />
    ...
</manifest>
```

Google Play Store membandingkan fitur yang diperlukan aplikasi dengan fitur yang tersedia di setiap perangkat pengguna untuk menentukan apakah aplikasi kompatibel dengan masing-masing perangkat. Jika perangkat tidak menyediakan semua fitur yang diperlukan aplikasi, pengguna tidak akan dapat menginstal aplikasi.

Akan tetapi, jika fungsi utama aplikasi tidak *membutuhkan* fitur perangkat, harus menyetel atribut `required` untuk `"false"` dan memeriksa fitur perangkat pada waktu proses. Jika fitur aplikasi tidak tersedia di perangkat saat ini, turunkan versi fitur aplikasi yang sesuai dengan baik. Misalnya, dapat menanyakan apakah fitur tersedia dengan memanggil `hasSystemFeature()` seperti ini:

```
PackageManager pm = getPackageManager();

if
(!pm.hasSystemFeature(PackageManager.FEATURE_SENSOR_COMPASS)) {
```

```
// This device does not have a compass, turn off  
the compass feature disableCompassFeature();  
}
```

Beberapa izin sistem secara implisit memerlukan ketersediaan fitur perangkat. Misalnya, jika aplikasi meminta izin untuk mengakses **BLUETOOTH**, secara implisit aplikasi membutuhkan fitur perangkat **FEATURE_BLUETOOTH**. dapat menonaktifkan pemfilteran berdasarkan fitur ini dan menyediakan aplikasi untuk perangkat tanpa Bluetooth dengan menyetel atribut **required** ke **"false"** dalam tag **<uses-feature>**. Untuk mengetahui informasi selengkapnya tentang fitur perangkat yang diperlukan secara implisit, baca **Izin yang Menerapkan Persyaratan Fitur**.

3. Versi Platform

Perangkat yang berbeda dapat menjalankan versi platform Android yang berbeda, seperti Android 4.0 atau Android 4.4. Setiap versi platform setelahnya sering menambahkan API baru yang tidak tersedia di versi sebelumnya. Untuk menunjukkan kumpulan API yang tersedia, setiap versi platform menentukan tingkat API. Misalnya, Android 1.0 adalah API level 1 dan Android 4.4 adalah API level 19. Tingkat API memungkinkan untuk mendeklarasikan versi minimum yang kompatibel dengan aplikasi, menggunakan tag manifes **<uses-sdk>** dan atribut **minSdkVersion**. Misalnya, API Penyedia Kalender ditambahkan di Android 4.0 (API level 14). Jika aplikasi tidak dapat berfungsi tanpa API ini, harus mendeklarasikan API level 14 sebagai versi minimum yang didukung aplikasi.

Atribut **minSdkVersion** mendeklarasikan versi minimum yang kompatibel dengan aplikasi dan atribut **targetSdkVersion** mendeklarasikan versi tertinggi yang telah dioptimalkan pada aplikasi.

Akan tetapi, perhatikan bahwa atribut dalam elemen **<uses-sdk>** digantikan oleh properti yang sesuai dalam file **build.gradle**. Jadi, jika menggunakan Android

Studio, harus menentukan nilai `minSdkVersion` dan `targetSdkVersion` di sini:

```
// Defines the minimum API level required to run the app.  
minSdkVersion 15  
  
// Specifies the API level used to test the app.  
targetSdkVersion 28  
...  
}  
}
```

Setiap versi Android setelahnya memberikan kompatibilitas untuk aplikasi yang dibuild menggunakan API dari versi platform sebelumnya, sehingga aplikasi harus selalu kompatibel dengan versi Android yang akan datang saat menggunakan Android API yang didokumentasikan.

Atribut **targetSdkVersion** tidak mencegah aplikasi diinstal di versi platform yang lebih tinggi dari nilai yang ditentukan, tetapi hal ini penting karena menunjukkan pada sistem apakah aplikasi harus mewarisi perubahan perilaku dalam versi yang lebih baru. Jika tidak mengupdate **targetSdkVersion** ke versi terbaru, sistem tersebut menganggap bahwa aplikasi memerlukan beberapa perilaku kompatibilitas dengan versi sebelumnya saat menjalankan versi terbaru. Misalnya, di antara perubahan perilaku di Android 4.4, alarm yang dibuat dengan API **AlarmManager** kini tidak tepat secara default sehingga sistem dapat menetapkan batch pada alarm aplikasi dan menghemat daya sistem. Akan tetapi, sistem akan mempertahankan perilaku API sebelumnya untuk aplikasi

jika tingkat API target lebih rendah dari "19". Namun, jika aplikasi menggunakan API yang ditambahkan dalam versi platform yang lebih baru, tetapi tidak mewajibkannya untuk fungsi utama, harus memeriksa tingkat API pada waktu proses dan menurunkan dengan baik fitur yang sesuai jika tingkat API terlalu rendah. Dalam hal ini, setting `minSdkVersion` ke nilai terendah yang memungkinkan untuk fungsi utama aplikasi, lalu bandingkan versi sistem saat ini, `SDK_INT`, dengan konstanta namakode dalam `Build.VERSION_CODES` yang sesuai dengan tingkat API yang ingin diperiksa.

4. Konfigurasi layar

Android berjalan di perangkat dengan berbagai ukuran, dari ponsel, tablet, dan TV. Untuk menggolongkan perangkat menurut jenis layarnya, Android menentukan dua karakteristik untuk setiap perangkat: ukuran layar (ukuran fisik layar) dan kepadatan layar (kerapatan fisik piksel pada layar, yang dikenal sebagai DPI). Untuk menyederhanakan konfigurasi yang berbeda, Android menggeneralisasi varian ini ke dalam grup yang membuatnya lebih mudah untuk ditargetkan:

- Empat ukuran umum: kecil, normal, besar, dan xlarge.
- Dan beberapa kepadatan umum: mdpi (sedang), hdpi (tinggi), xhdpi (ekstra tinggi), xxhdpi (ekstra-ekstra tinggi), dan lainnya.

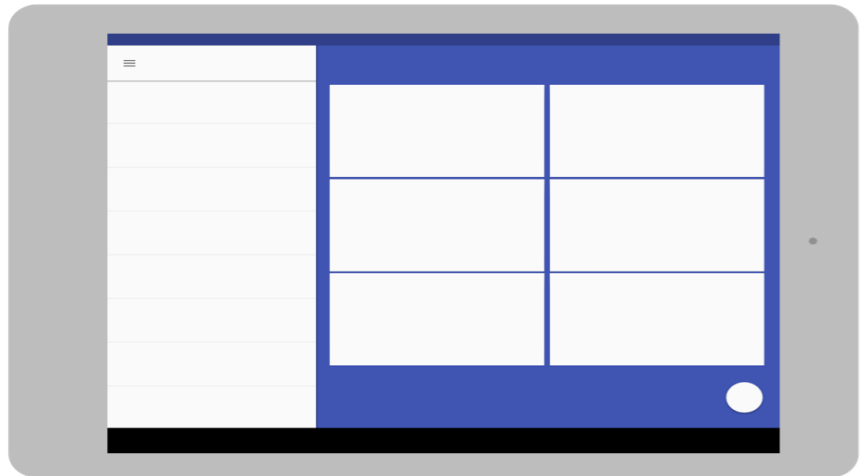
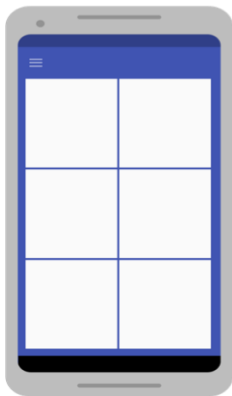
Secara default, aplikasi kompatibel dengan semua ukuran dan kepadatan layar, karena sistem membuat penyesuaian yang tepat pada tata letak UI dan resource gambar yang diperlukan untuk setiap layar. Akan tetapi, harus mengoptimalkan pengalaman pengguna untuk setiap konfigurasi layar dengan menambahkan tata letak khusus untuk ukuran layar yang berbeda dan gambar bitmap yang dioptimalkan untuk kepadatan layar yang umum. Untuk mengetahui informasi tentang cara membuat resource alternatif untuk berbagai layar dan cara membatasi aplikasi ke ukuran layar tertentu bila diperlukan. Android berjalan pada beragam perangkat yang memiliki berbagai ukuran layar dan kepadatan piksel. Sistem akan melakukan penskalaan dasar dan mengubah ukuran untuk menyesuaikan antarmuka pengguna ke beragam layar, tetapi masih ada banyak pekerjaan yang harus dilakukan untuk memastikan User Interface beradaptasi dengan tepat ke setiap jenis layar.



Gambar 19 Konfigurasi Layar

Ukuran layar adalah ruang terlihat yang disediakan untuk UI aplikasi . Ukuran layar yang diketahui oleh aplikasi bukanlah ukuran layar perangkat yang sebenarnya—ukuran ini mempertimbangkan orientasi layar, dekorasi sistem (seperti menu navigasi), dan perubahan konfigurasi jendela (seperti saat pengguna mengaktifkan mode multi-jendela. Secara default, Android mengubah ukuran layout aplikasi agar sesuai dengan layar saat ini. Untuk memastikan layout dapat diubah ukurannya dengan baik bahkan ketika terdapat perbedaan kecil dalam ukuran layar, harus mengimplementasikan layout dengan mempertimbangkan fleksibilitas. Prinsip pokok yang harus diikuti adalah menghindari hard-coding untuk posisi dan ukuran komponen UI . Sebagai gantinya, buat pengaturan ukuran tampilan untuk membesar dan menetapkan posisi tampilan relatif terhadap tampilan induk atau tampilan sejenis lainnya sehingga urutan yang diinginkan dan ukuran relatif tetap sama dengan layout yang semakin besar.

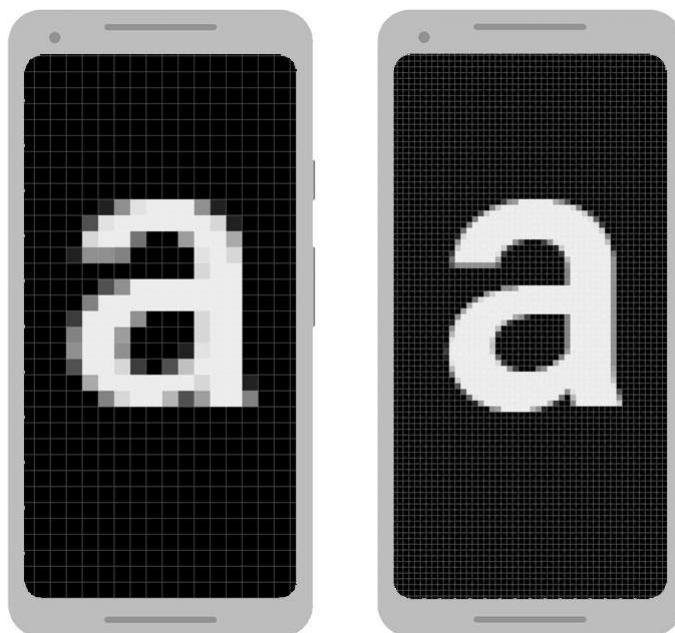
Layout fleksibel sangatlah penting, tetapi juga harus mendesain layout berbeda yang mengoptimalkan pengalaman pengguna untuk ruang yang tersedia pada beragam perangkat seperti ponsel dan tablet. Jadi Android mengizinkan untuk menyediakan file layout alternatif yang diterapkan sistem saat runtime berdasarkan ukuran layar perangkat saat ini.



Gambar 20 Aplikasi yang sama menggunakan layout berbeda untuk ukuran layar yang berbeda

Karena layout harus direntangkan agar sesuai dengan ukuran layar, demikian juga semestinya bitmap yang disematkan ke setiap tampilan layout. Namun, merentangkan bitmap biasa secara sembarangan bisa menghasilkan artefak penskalaan yang aneh dan gambar yang tidak simetris. Untuk mengatasi ini, Android mendukung bitmap nine-patch tempat menentukan area piksel kecil yang dapat direntangkan—bagian gambar yang lain tidak diubah.

Kepadatan piksel adalah jumlah piksel dalam area fisik layar dan sering disebut sebagai dpi (dots per inch). Ini berbeda dengan resolusi, yang merupakan jumlah total piksel pada layar.



Gambar 21 Pembesaran Gambart

Aplikasi mencapai "independensi kepadatan" bila mempertahankan ukuran fisik (dari sudut png pengguna) desain UI saat ditampilkan pada layar dengan kepadatan piksel yang berbeda (seperti yang ditunjukkan pada gambar 2). Menjaga independensi kepadatan adalah hal yang penting karena, tanpanya, elemen UI (seperti tombol) mungkin terlihat lebih besar pada layar berkepadatan rendah dan lebih kecil pada layar berkepadatan tinggi (karena ketika piksel lebih besar—seperti yang ditunjukkan pada gambar 2— beberapa piksel bisa meluber).

Sistem Android membantu mencapai independensi kepadatan dengan menyediakan *piksel berkepadatan-independen* (dp atau dip) sebagai unit pengukuran yang harus gunakan sebagai ganti piksel (px).

Untuk memastikan gambar terlihat seoptimal mungkin pada semua layar, harus menyediakan bitmap alternatif agar pas dengan setiap kepadatan layar. Misalnya, jika aplikasi hanya menyediakan bitmap untuk layar berkepadatan menengah (mdpi), Android akan memperbesarnya ketika ditampilkan pada layar berkepadatan-tinggi sehingga gambar mempergunakan ruang fisik yang sama di layar. Ini bisa menyebabkan artefak penskalaan yang terlihat dalam bitmap. Jadi aplikasi harus menyertakan bitmap alternatif pada resolusi yang lebih tinggi.

Untuk jenis gambar sederhana (biasanya ikon), bisa menghindari pembuatan gambar terpisah untuk setiap kepadatan dengan menggunakan grafik vektor. Karena grafik vektor menetapkan ilustrasi dengan jalur garis geometris, bukan piksel, mereka bisa digambar dalam ukuran apa pun tanpa artefak penskalaan. Rekomendasi tersebut berlaku untuk semua faktor bentuk Android, tetapi jika ingin membuat aplikasi untuk Wear OS, Android TV, Android Auto, atau perangkat Chrome OS, harus melakukan lebih banyak pengaturan.

Masing-masing perangkat ini memiliki model interaksi pengguna mereka sendiri yang harus diakomodasi aplikasi. Dalam beberapa kasus, seperti Wear OS, harus memikirkan ulang pengalaman pengguna aplikasi dan membuat aplikasi yang khusus untuk perangkat tersebut. Dan untuk mendukung perangkat Chrome OS (seperti Google Pixelbook), mungkin hanya memerlukan sedikit

modifikasi pada aplikasi saat ini untuk mendukung interaksi keyboard/mouse dan layar yang jauh lebih besar.

5. Perangkat yang Mendukung Potongan Layar



Gambar 22 Potongan Layar

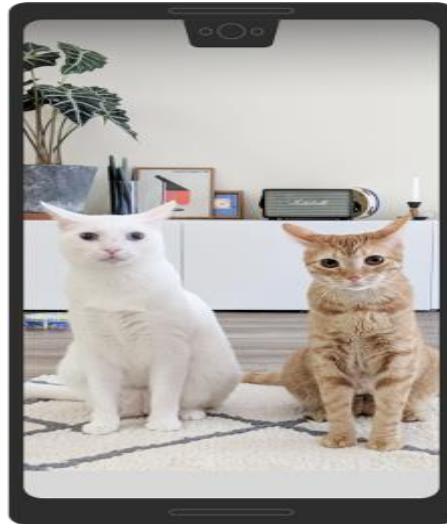
Potongan layar adalah area di beberapa perangkat yang meluas ke permukaan tampilan untuk memberikan pengalaman dari tepi ke tepi, sekaligus memberikan ruang bagi sejumlah sensor penting di bagian depan perangkat. Android secara resmi mendukung potongan layar di perangkat yang menjalankan Android 9 (API level 28) dan yang lebih tinggi. Perhatikan bahwa produsen perangkat juga dapat memilih untuk mendukung potongan layar di perangkat yang menjalankan Android 8.1 atau yang lebih rendah.

Topik ini menjelaskan cara menerapkan dukungan untuk perangkat yang memiliki potongan layar, termasuk cara menangani *area potongan*--yakni, persegi panjang dari tepi ke tepi di permukaan tampilan yang berisi potongan.

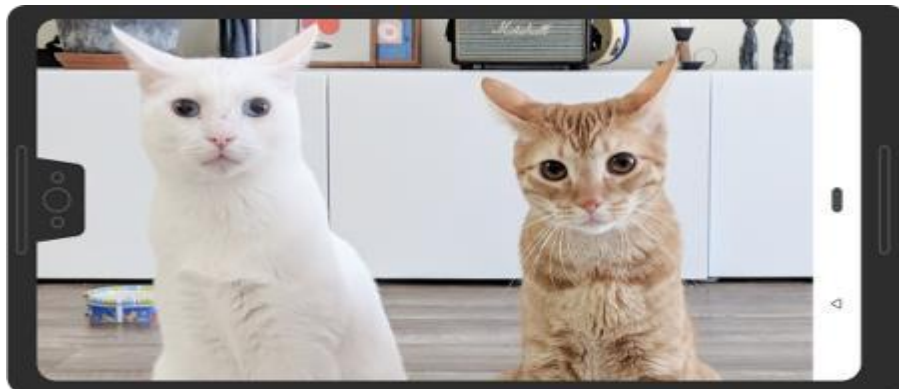
Untuk memastikan konsistensi dan kompatibilitas aplikasi, perangkat yang menjalankan Android 9 harus memastikan perilaku potongan layar berikut:

- Satu tepi dapat berisi maksimal satu potongan.
- Perangkat tidak boleh memiliki lebih dari dua potongan.

- Perangkat tidak boleh memiliki potongan layar di tepi panjangnya.
- Dalam orientasi potret tanpa tanda khusus yang ditetapkan, status bar harus meluas hingga setidaknya setinggi potongan.
- Secara default, dalam orientasi lanskap atau layar penuh, seluruh area potongan harus diberi letterbox.



Gambar 23 Contoh Potongan layar potrait



Gambar 24 Contoh Potongan Layar Landscape

Perhatikan bahwa potongan di sudut dianggap berada di tepi pendek, sehingga berlaku perilaku yang sama:

Selain membatasi ketersediaan aplikasi berdasarkan karakteristik perangkat, mungkin harus membatasi ketersediaan aplikasi karena alasan bisnis atau hukum. Misalnya, aplikasi yang menampilkan jadwal kereta untuk Kereta Bawah Tanah London kemungkinan tidak akan berguna bagi pengguna di luar Inggris Raya. Untuk jenis situasi ini, Google Play Store menyediakan opsi pemfilteran di Konsol Play yang memungkinkan mengontrol ketersediaan aplikasi karena alasan non-teknis seperti lokal atau operator nirkabel pengguna. Pemfilteran untuk kompatibilitas teknis (seperti komponen hardware yang diperlukan) selalu didasarkan pada informasi yang terdapat dalam file APK . Akan tetapi, memfilter karena alasan non-teknis (seperti lokal geografis) selalu ditangani di Konsol Google Play.

D. LEMBAR KERJA

Diskusikan dan kerjakan dalam kelompok masing-masing, beberapa pertanyaan berikut ini.

1. Buatlah analisa perkembangan sistem operasi android dari android 1.0 hingga sekarang beserta penambahan fitur masing-masing versi android tersebut (Bisa dalam bentuk bagan, table atau bentuk lain sesuai kreativitas tim Anda).
2. Lakukan Analisa fitur perangkat keras dan perangkat lunak pada smartphone Anda sesuai contoh penjelasan modul, termasuk versi Android, Processor, dan RAM (Bisa dalam bentuk bagan, table atau bentuk lain sesuai kreativits tim Anda)
3. Klasifikasikan mana yang termasuk kompatibilitas perangkat dan mana yang termasuk kompatibilitas aplikasi pada smartphone android anda. Bisa dibuat dalam bentuk tabel atau bagan atau bentuk lain beserta penjelasan dari setiap kompatibilitas tersebut.

BAB II ARSITEKTUR APLIKASI ANDROID

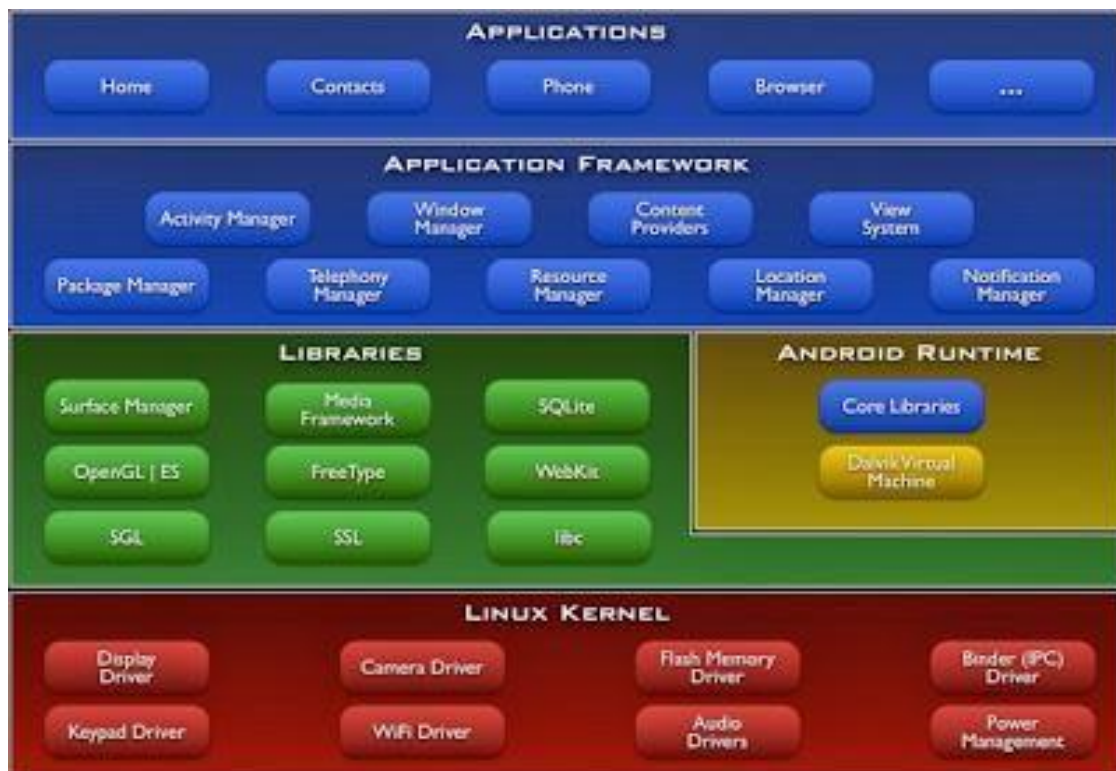
TUJUAN PEMBELAJARAN

Setelah melakukan diskusi kelompok dengan metode windows shopping, peserta dapat:

1. Melakukan Analisa Lapisan arsitektur sistem operasi android
2. Melakukan Analisa Komponen Aplikasi android
3. Melakukan Analisa penggunaan inten untuk mengaktifkan komponen dalam aplikasi android

A. Arsitektur Sistem Operasi Android

Arsitektur Android dipandang sebagai sebuah tumpukan software. Setiap lapisan dari tumpukan ini menghimpun beberapa program yang mendukung fungsi-fungsi spesifik dari sistem operasi. Berikut ini susunan dari lapisan-lapisan tersebut jika dilihat dari lapisan dasar hingga lapisan teratas.



Berikut ini penjelasan terperinci dari setiap lapisan arsitektur android.

1. Linux Kernel

Tumpukan paling bawah pada arsitektur Android adalah kernel. Google menggunakan kernel Linux versi 2.6 untuk membangun sistem Android, yang mencakup memory management, security setting, power management, dan beberapa driver hardware.

Kernel berperan sebagai abstraction layer antara hardware dan keseluruhan software.

Sebagai contoh, HTC G1 dilengkapi dengan kamera. Kernel Android terdapat driver kamera yang memungkinkan pengguna mengirimkan perintah ke hardware kamera.

2. Android Runtime

Lapisan setelah kernel Linux adalah Android Runtime. Android Runtime ini berisi CORE LIBRARIES dan DALVIK VIRTUAL MACHINE.

Core Libraries mencakup serangkaian inti library Java, artinya Android menyertakan satu set library-library dasar bahasa pemrograman Java.

Dalvik adalah Java Virtual Machine yang memberi kekuatan pada sistem Android. Dalvik VM ini di optimalkan untuk telepon seluler.

Setiap aplikasi yang berjalan pada Android berjalan pada prosesnya sendiri, dengan instance dari Dalvik Virtual Machine. Dalvik telah dibuat sehingga sebuah piranti yang memakainya dapat menjalankan multi Virtual Machine dengan efisien. Dalvik VM dapat mengeksekusi file dengan format Dalvik Executable (.dex) yang telah dioptimasi untuk menggunakan minimal memory footprint. Virtual Machine ini register-based, dan menjalankan class-class yang dicompile menggunakan compiler java yang kemudian ditransformasi menjadi format .dex menggunakan "dx" tool yang telah disertakan.

Dalvik VM menggunakan kernel Linux untuk menjalankan fungsi fungsi seperti threading dan low-level memory management.

3. Libraries

Bertempat di level yang sama dengan Android Runtime adalah Libraries. Android menyertakan satu set library-library dalam bahasa C/C++ yang digunakan oleh berbagai komponen yang ada pada sistem Android. Kemampuan ini dapat diakses oleh programmer melewati Android application framework. Sebagai contoh Android mendukung pemutaran format audio, video, dan gambar.

Berikut ini beberapa core library tersebut :

- **System C Library**, Diturunkan dari implementasi standard C system library (libc) milik BSD, dioptimasi untuk piranti embedded berbasis Linux
- **Media Libraries**, Berdasarkan Packet Video's OpenCORE; library-library ini mendukung playback dan recording dari berbagai format audio dan video populer, meliputi MPEG4, H.264, MP3, AAC, AMR, JPG, dan PNG
- **Surface Manager**, Mengatur akses pada display dan lapisan composites 2D dan 3D graphic dan berbagai aplikasi
- **Lib Web Core**, Web browser engine modern yang mensupport Android browser maupun embeddable web view
- **SGL**, Teh underlying 2D Graphics Engine
- **3D Libraries**, Implementasi berdasarkan OpenGL ES 1.0 APIs; library ini menggunakan hardware 3D acceleration dan highly optimized 3D software restirezer.
- **FreeType**, Bitmap dan vector font rendering
- **SQLite**, Relational databse engine yang powerful dan ringan tersedia untuk semua aplikasi

4. Aplication Framework

Lapisan selanjutnya adalah Application Framework yang mencakup program untuk mengatur fungsi-fungsi dasar smartphone. Application Framework merupakan serangkaian tool dasar seperti alokasi resource smartphone,

aplikasi telepon, pergantian antar - proses atau program, dan pelacakan lokasi fisik. Pengembang aplikasi memiliki aplikasi penuh kepada tool-tool dasar tersebut, dan memanfaatkannya untuk menciptakan aplikasi yang lebih kompleks. Programmer mendapatkan akses penuh untuk memanfaatkan API-API (Android Protocol Interface) yang juga digunakan core applications. Arsitektur aplikasi didesain untuk menyederhanakan pemakaian kembali komponen-komponen, setiap aplikasi dapat menunjukkan kemampuannya dan aplikasi lain dapat memakai kemampuan tersebut.

Mekanisme yang sama memungkinkan pengguna mengganti komponen-komponen yang dikehendaki. Di dalam semua aplikasi terdapat servis dan sistem yang meliputi :

- Satu set Views yang dapat digunakan untuk membangun aplikasi meliputi lists, grids, text boxes, buttons, dan embeddable web browser
- Content Providers, yang memungkinkan aplikasi untuk mengakses data dari aplikasi lain (mislnya contacts), atau untuk membagi data yang dimilikinya.
- Resource Manger, menyediakan akses ke non-code resources misalnya localized strings, graphics, dan layout files
- Notification Manager, yang memungkinkan semua aplikasi untuk menampilkan custom alerts pada status bar
- Activity Manager, yang manage life cycle of dari aplikasi dan menyediakan common navigation backstack

5. Application

Susunan lapisan teratas terdapat aplikasi itu sendiri. Dalam lapisan inilah ditemukan fungsi-fungsi dasar smartphone seperti menelepon dan mengirim pesan singkat, menjalankan web browser, mengakses daftar konten, dan lain-lain. Bagi rata-rata pengguna, lapisan inilah yang paling sering diakses. Pengguna aplikasi android mengakses fungsi-fungsi dasar tersebut melalui user interface.

B. Komponen Aplikasi Android

Komponen aplikasi android adalah suatu bagian yang sangat penting dari suatu aplikasi Android karena dengan komponen-komponen inilah suatu aplikasi android dapat berjalan dengan baik. Komponen-komponen ini dikendalikan oleh *AndroidManifest.xml* yang mendeskripsikan setiap komponen dan bagaimana komponen itu berinteraksi.

Berikut ini adalah 4 komponen utama yang dapat digunakan dalam sebuah aplikasi Android :

Komponen	Deskripsi
Activities	Suatu komponen yang mengendalikan User Interface dan menangani interaksi pengguna ke layar smart phone.
Services	Suatu komponen yang menangani proses di background yang terhubung dengan aplikasi.
Broadcast Receivers	Suatu komponen yang menangani komunikasi antara Sistem Operasi Android dengan aplikasi.
Content Providers	Suatu komponen yang menangani data dan masalah manajemen basis data.

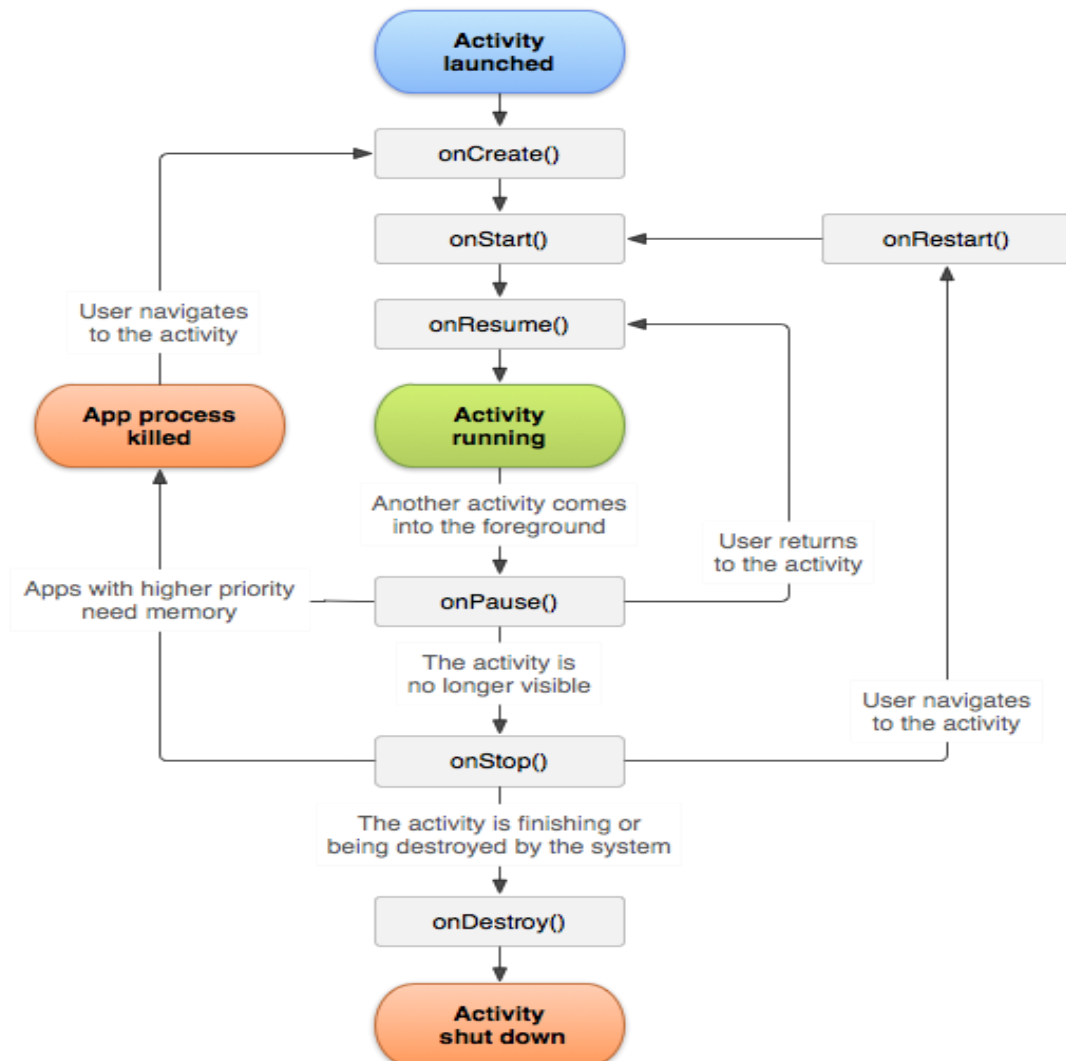
1. Activities

Sebuah activity merepresentasikan sebuah layar dengan User Interface. Dalam artian, activity yang melakukan aksi pada layar, Contohnya: Ada halaman utama lalu ada halaman kedua, pada halaman utama ada tombol next menuju halaman kedua. Halaman utama adalah sebuah activity yang berbeda dengan halaman kedua. Jika sebuah aplikasi memiliki lebih dari 1 *activity*, maka salah satu dari *activity* tersebut harus ditandai sebagai *activity* yang ditampilkan secara default ketika aplikasi dijalankan.

Activity adalah komponen yang dapat dilihat oleh pengguna, sehingga pengguna dapat berinteraksi dengan aplikasi. Bila dibandingkan dengan aplikasi web dan desktop, *Activity* pada aplikasi android sama seperti halaman pada aplikasi web dan *form* pada aplikasi desktop

Aplikasi Android	Aplikasi Web	Aplikasi Desktop
<i>Activity</i>	Halaman	<i>Form</i>

Activity memiliki siklus hidup (*lifecycle*) yang merupakan kondisi yang akan dialami saat diciptakan sampai dihancurkan. Ada beberapa kondisi yang akan dialami oleh *Activity* seperti yang tergambar pada *flow chart* berikut ini.



Penjelasannya:

- onCreate() adalah kondisi awal saat *Activity* baru diciptakan, biasanya dilakukan inisialisasi pada tahapan ini.
- onStart() adalah saat *Activity* dimulai
- onResume() adalah saat *Activity* dibuka kembali, biasanya dieksekusi setelah onPause()
- onPause() akan dipanggil saat ada *Activity* lain yang terbuka.
- onStop() adalah kondisi saat *Activity* tidak ditampilkan dilayar (biasanya saat pengguna menekan tombol Home).
- onRestart() adalah kondisi saat *Activity* kembali dibuka oleh pengguna.
- onDestroy() adalah kondisi saat *Activity* dihancurkan pada memori

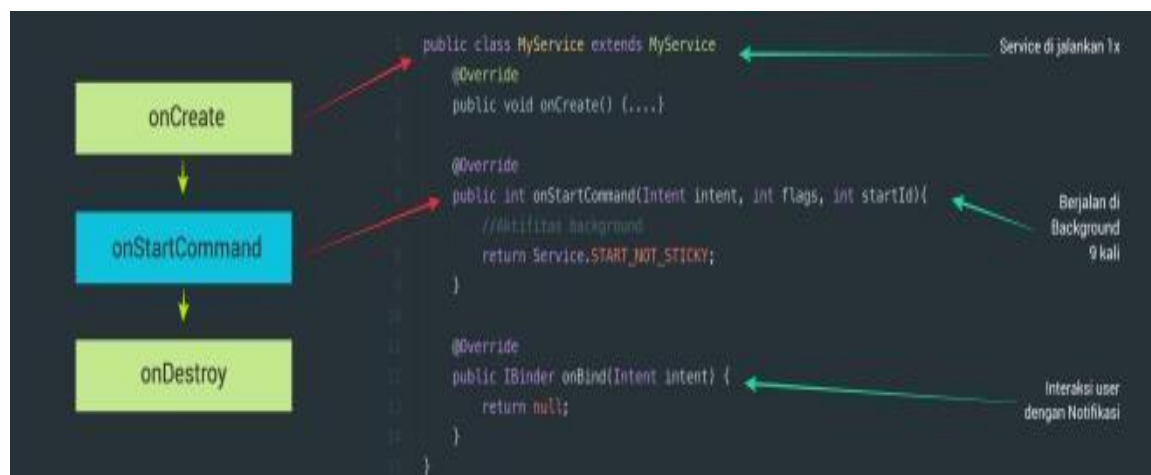
2. Services

Service adalah komponen yang berjalan di background untuk melakukan operasi yang berjalan panjang, Contohnya: sebuah service dapat memainkan music di background sementara pengguna dalam aplikasi yang berbeda atau service dapat mengambil data dalam jaringan tanpa menghalangi interaksi user dengan sebuah activity. *Service* merupakan komponen aplikasi yang menjalankan operasi tertentu yang bisa berjalan lama pada sistem Android, biasanya *service* itu berjalan di latar belakang. *Service* ini tidak menyediakan antarmuka pengguna (UI).

Gambaran dari penggunaan services yaitu aplikasi yang ada di perangkat android tetap bekerja walaupun aplikasi tersebut tidak terlihat berjalan di depan layar, contohnya download file, upload foto, dan juga memutar musik. Komponen aplikasi yang mengurusnya inilah yang dinamakan dengan services, tapi service tidak memiliki interface, dan juga menjalankan aplikasi berdasarkan prioritas dari aplikasi itu sendiri. Untuk menggunakan services, perlu membuat atau menggunakan subclass dari services itu sendiri, yaitu:

- onStartCommand
- onCreate
- onBind
- onDestroy

Misalnya ingin mengirim notifikasi (contohnya: *chatting*) dari server ke aplikasi sebanyak 10 kali, maka berikut cara kerja *Services* di Android:



Penjelasan:

1. Yang dilakukan *Services* pertama kali adalah menerima notifikasi pertama dan memulai menjalankan *onCreate* untuk *services*nya.
2. Jika telah berhasil, maka notifikasi selanjutnya berjalan secara background task menggunakan *onStarCommand*.
3. Sampai notifikasi didapatkan, dan ketika user berinteraksi dengan notifikasi tersebut, maka *onBind* yang dipanggil untuk memproses interaksi tersebut.

Contoh lain dari implementasi *services* yaitu pada aplikasi facebook. Dimana *services* akan terus berjalan secara background walaupun pengguna berinteraksi atau tidak saat adanya notifikasi *friend request*. Contoh interaksi langsung dengan *services* adalah saat *accept friend request*. Ada banyak aplikasi yang mengimplementasikan *services* ini, terutama aplikasi yang memiliki fitur notifikasi, *chatting*, dan sejenisnya.

Ada dua macam *service* pada Android, yaitu :

- **Started**

Service yang komponen aplikasinya dimulai dengan memanggil *startService()*. *Service* ini akan berjalan selama belum dimatikan atau proses yang dijalankan selesai. *Service* akan tetap berjalan walaupun komponen yang lain dimatikan oleh sistem Android. *Service* ini digunakan untuk melakukan proses yang tidak memberikan nilai balik ke komponen yang memanggil. Contohnya seperti mengunduh atau mengunggah berkas.

- **Bound**

Service jenis ini merupakan tipe *service* yang dijalankan oleh komponen lain, namun saling mengikat satu sama lain. Hubungan yang terjadi antar kedua komponen tersebut seperti *client-server*. Bisa saling menerima *response* dan menerima *request* yang ada. Pada *service* ini dimungkinkan terjadi proses IPC (*Interprocess Communication*). *Service* ini akan tetap berjalan di background selama masih ada komponen lain yang mengikatnya. Jika tidak, maka *Service* akan dimatikan oleh sistem. Aplikasi pemutar musik merupakan salah satu jenis aplikasi yang mengimplementasikan *Service* jenis ini. Perlu diingat, *Service* berjalan di *Thread* Utama dari proses hostingnya. *Service* tidak membuat *Thread* sendiri dan tidak berjalan di proses terpisah. Kecuali jika anda menentukannya.

Jika *service* akan melakukan pekerjaan yang banyak membutuhkan *CPU* atau operasi pemblokiran (seperti pemutaran MP3 atau jaringan), maka memerlukan pembuatan *Thread* baru dalam layanan untuk melakukan pekerjaan itu.

Penggunaan *Thread* terpisah, akan mengurangi risiko kesalahan seperti aplikasi tidak merespons (Application Not Responding/ANR) dan *Thread* utama aplikasi bisa terus disediakan untuk interaksi pengguna.

Sebagaimana dengan *activity* dan komponen lainnya, *service* harus dideklarasikan dalam file *AndroidManifest.xml*. Untuk mendeklarasikan *service*, tambahkan elemen `<service>` sebagai anak dari elemen `<application>` .

Contohnya :

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ...>

  <application ... >
    ...
    <service
      android:name=".DownloadIntentService"
      android:exported="false"/>
    </application>
  </manifest>
```

Untuk memblokir akses ke *service* dari aplikasi lainnya, deklarasikan *service* sebagai privat. Caranya, lakukan pengaturan atribut *android:exported* ke *false*. Ini akan menghentikan aplikasi lain dari memulai *service*, bahkan bila menggunakan *Intent* eksplisit.

Started Services

1. Komponen aplikasi seperti *activity* memanggil *startService()* dan meneruskannya di *Intent*. Dalam hal ini *Intent* menetapkan *service* dan menyertakan data yang akan digunakan oleh *service*.
2. Sistem akan memanggil metode *onCreate()* dan *callback* lainnya yang sesuai di *thread* utama. Selanjutnya mengimplementasikan *callback* tersebut dengan perilaku yang sesuai, seperti membuat *thread* sekunder yang akan digunakan.
3. Sistem akan memanggil metode *onStartCommand()*, dengan meneruskan *Intent* yang disediakan oleh klien di langkah 1. (Klien dalam konteks ini adalah komponen aplikasi yang memanggil *service*).

Setelah dimulai, *service* bisa berjalan di latar belakang tanpa dibatasi waktu, bahkan jika komponen yang memulainya telah dimusnahkan. Biasanya, *service* yang dimulai menjalankan operasi tunggal dan tidak mengembalikan hasil ke pemanggil. Misalnya, layanan dapat mengunduh atau mengunggah file melalui jaringan. Bila operasi selesai, *service* harus berhenti sendiri dengan memanggil *stopSelf()*, atau komponen lain bisa menghentikannya dengan memanggil *stopService()*.

Misalnya, anggaplah aktivitas perlu menyimpan data ke database online. Aktivitas akan memulai layanan pendamping dengan meneruskan *Intent* ke *startService()*. Layanan menerima maksud di *onStartCommand()*, menghubungkan ke Internet, dan menjalankan transaksi database.

Bila transaksi selesai, layanan akan menggunakan *stopSelf()* untuk menghentikan dirinya sendiri dan dimusnahkan. (Ini adalah contoh layanan yang ingin Anda jalankan di *thread pekerja/worker thread*, sebagai ganti *thread utama*).

IntentService

Sebagian besar *service* yang dimulai tidak perlu menangani beberapa permintaan secara bersamaan, dan jika *service* melakukannya, maka akan mengakibatkan Skenario multi-threading yang berbahaya. Karena itu, sebaiknya Anda mengimplementasikan *service* menggunakan kelas *IntentService*.

IntentService adalah subkelas yang berguna dari *Service* :

- *IntentService* secara otomatis menyediakan *thread pekerja/worker thread* untuk menangani *Intent*.
- *IntentService* menangani beberapa kode *boilerplate* yang diperlukan layanan umum (seperti memulai dan menghentikan layanan).
- *IntentService* bisa membuat antrian pekerjaan yang meneruskan satu *intent* untuk setiap kalinya ke implementasi *onHandleIntent()*, sehingga Anda tidak perlu mengkhawatirkan multi-threading.

Bound Services

“*Bound*” *Services* bila komponen aplikasi mengikatnya dengan memanggil *bindService()*. *Bound Services* menawarkan antarmuka *client-server* yang memungkinkan komponen berinteraksi dengan *service*, mengirim *request*, dan mendapatkan *response*, kadang-kadang menggunakan komunikasi interproses (IPC) untuk mengirim dan menerima informasi di seluruh proses. *Service* ini hanya berjalan selama komponen aplikasi terikat padanya. Beberapa komponen bisa diikat ke *service* sekaligus, namun bila semuanya telah dilepas, *service* akan dimusnahkan. *Service* ini umumnya tidak mengizinkan memulai komponen dengan memanggil *startService()*.

Untuk mengimplementasikan *Bound Services*, definisikan antarmuka yang menetapkan cara klien bisa berkomunikasi dengan *service*. Antarmuka ini, yang dikembalikan *service* Anda dari metode callback *onBind()*, harus berupa implementasi *IBinder*. Untuk mengambil antarmuka *IBinder*, komponen aplikasi klien memanggil *bindService()*. Setelah klien menerima *IBinder*, klien berinteraksi dengan *service* melalui antarmuka itu. Ada sejumlah cara untuk mengimplementasikan *Bound Services*, dan implementasi tersebut lebih rumit daripada *Started Services*. Untuk detail selengkapnya tentang *Bound Services*, lihat *Bound Services*.

Mengikat ke Service

Untuk mengikat ke *service* yang dideklarasikan di file *AndroidManifest.xml* dan diimplementasikan oleh komponen aplikasi, gunakan *bindService()* dengan *Explicit Intent*.

Perhatian: Jangan gunakan *Implicit Intent* untuk mengikat ke *service*. Melakukannya adalah bahaya keamanan, karena tidak bisa memastikan

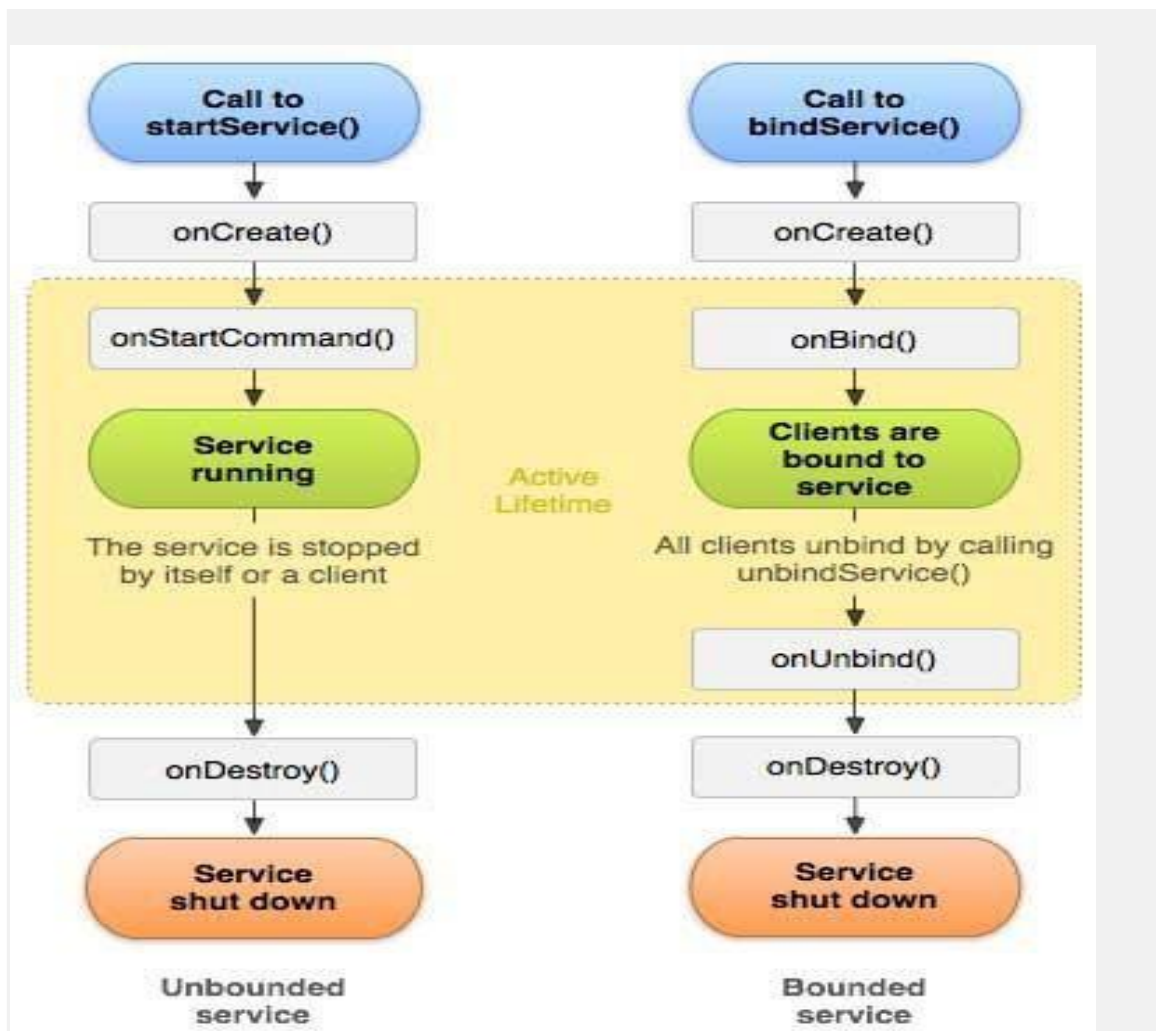
service yang akan merespons *Intent* tersebut, dan pengguna tidak bisa melihat *service* mana yang dimulai. Mulai dengan Android 5.0 (*API level 21*), sistem membuat *exception* jika memanggil *bindService()* dengan *Implicit Intent*.

Life Cycle Service

Daur hidup *service* lebih sederhana daripada *activity*. Akan tetapi, ini jauh lebih penting karena memperhatikan dari dekat cara *service* dibuat dan dimusnahkan. Karena tidak memiliki *UI*, *service* bisa terus berjalan di latar belakang tanpa diketahui pengguna, bahkan jika pengguna beralih ke aplikasi lain. Ini menghabiskan sumber daya dan menguras baterai.

Seperti *activity*, *service* memiliki metode callback daur hidup yang bisa diimplementasikan untuk memantau perubahan keadaan *service* dan melakukan pekerjaan pada waktu yang sesuai. Berikut kerangka *service* yang memperagakan setiap metode daur hidup *Started Service* dan *Bound Service*. *Bound Service* hanya tersedia untuk menyajikan komponen aplikasi yang terikat padanya, sehingga bila tidak ada lagi komponen yang diikat ke *service* tersebut, sistem akan memusnahkannya. *Bound Service* tidak perlu dihentikan secara eksplisit seperti halnya *Started Service* (menggunakan *stopService()* atau *stopSelf()*).

Gambar berikut ini adalah menampilkan perbedaan daur hidup dari keduanya



Foreground Service

Walaupun sebagian besar layanan berjalan di latar belakang, sebagian lagi ada yang berjalan di latar depan (*Foreground*). *Foreground Service* adalah *service* yang diketahui pengguna, jadi ini bukan layanan yang akan dimatikan sistem bila memori tinggal sedikit. Misalnya, pemutar musik yang memutar musik dari *service* harus disetel untuk berjalan di latar depan, karena pengguna mengetahui operasinya. Notifikasi di bilah status dapat menunjukkan lagu saat ini dan memungkinkan pengguna meluncurkan aktivitas untuk berinteraksi dengan pemutar musik.

Untuk meminta agar layanan berjalan di latar depan, dengan cara memanggil *startForeground()* sebagai ganti *startService()*. Metode ini menggunakan dua parameter: integer yang secara unik mengidentifikasi notifikasi dan Notification untuk bilah status. Notifikasi ini sedang berlangsung, artinya tidak bisa ditutup. Notifikasi tetap berada di bilah status hingga *service* dihentikan atau dibuang dari latar depan. Untuk membuang layanan dari latar depan, dengan cara memanggil *stopForeground()*. Metode ini memerlukan *boolean*, yang menunjukkan apakah akan membuang notifikasi bilah status atau tidak. Metode ini tidak menghentikan *service*. Akan tetapi, jika Anda menghentikan *service* sewaktu masih berjalan di latar depan, maka notifikasi juga akan dibuang.

Scheduled services

Untuk *API* level 21 dan yang lebih tinggi, bisa meluncurkan *service* menggunakan *JobScheduler API*. Untuk menggunakan *JobScheduler*, perlu mendaftarkan tugas dan menetapkan persyaratannya untuk jaringan dan pengaturan waktu. Sistem menjadwalkan tugas untuk dieksekusi di waktu yang tepat. Antarmuka *JobScheduler* menyediakan banyak metode untuk mendefinisikan ketentuan eksekusi layanan.

3. Broadcast Receivers

Broadcast Receiver adalah sebuah komponen Android yang berfungsi untuk melakukan broadcast pesan dari aplikasi lain atau dari sistem itu sendiri. Pesan yang di broadcast disebut event atau intent. Aplikasi melakukan broadcast untuk memberi tahu aplikasi lainnya bahwa beberapa data telah di unduh ke perangkat yang tersedia, lalu penerima broadcast akan menerima pesan untuk melakukan transfer informasi

Berikut ini adalah cara kerja Broadcast Receiver ketika dijalankan, yaitu :

- Register Broadcast Receiver
- Broadcast Intent
- Android mengirimkan Intent ke penerima yang telah terdaftar dengan memanggil method `onReceive()`
- Event diterima oleh method `onReceive()`

Ada beberapa langkah penting untuk membuat Broadcast Receiver, yaitu:

- Creating Broadcast Receiver
- Registering Broadcast Receiver

Sebuah broadcast sesungguhnya adalah pesan yang dibungkus menggunakan objek Intent. Broadcast bisa dibuat secara *implicit* atau *explicit*.

- Sebuah ***implicit broadcast*** adalah pesan yang tidak hanya ditujukan pada aplikasi kita saja tapi siapapun. Untuk menerima pesan *implicit* ini, kita perlu menggunakan sebuah IntentFilter dan mendeklarasikannya di dalam manifest. Kita perlu melakukan langkah ini karena sistem operasi Android akan memberikan pesan broadcast pada setiap aplikasi yang sudah memiliki intent filter untuk pesan atau event tertentu. Karena cara kerjanya ini, implicit broadcast tidak menargetkan aplikasi tertentu. Contoh implicit intent misalnya adalah event saat ada SMS masuk.
- Sebuah ***explicit broadcast*** adalah pesan yang ditujukan untuk aplikasi-aplikasi tertentu karena sudah menyiapkan suatu komponen. Hal ini terjadi karena ada atribut target yang menyebutkan package name atau nama kelas dari suatu aplikasi.

Tujuan pembuatan pesan broadcast dari dalam aplikasi yaitu untuk memungkinkan aplikasi tersebut bereaksi terhadap suatu event yang terjadi di dalamnya. Misalnya ketika pengguna melakukan suatu aksi, dan ingin melakukan suatu proses setelah aksi tersebut terjadi. Ada tiga cara untuk mengirimkan broadcast, yakni:

1. Method **`sendOrderedBroadcast`**, kita bisa memastikan pesan broadcast untuk dikirimkan hanya pada satu receiver. Setiap broadcast secara bergantian bisa mengirimkan data ke broadcast sesudahnya atau menyelesaikan prosesnya di kelas receiver yang menerima.
2. Method **`sendBroadcast`** mirip dengan method di atas, tapi dengan satu perbedaan. Semua broadcast receiver akan menerima pesan broadcast dan tidak bergantung pada broadcast lain.
3. Method **`LocalBroadcastManager.sendBroadcast`** hanya akan mengirimkan pesan broadcast ke receiver yang terdapat di dalam aplikasi yang sama dan tidak mengumumkannya ke aplikasi lain.

Broadcast Receivers merespon terhadap pesan broadcast dari aplikasi lain atau dari system, Contohnya: aplikasi juga dapat melakukan broadcast agar aplikasi lain tahu bahwa beberapa aplikasi yang telah di download ke dalam perangkat dan dapat mereka gunakan. Di sinilah fungsi Broadcast Receivers yang berfungsi mengambil alih komunikasi seperti ini dan menginisiasi aksi yang tepat.

Hal-hal yang perlu diperhatikan terkait Broadcast receiver adalah sebagai berikut:

- Jangan mengirimkan data sensitif melalui implicit broadcast karena aplikasi apapun yang menunggu data tersebut bisa menerimanya. Kita bisa mencegah hal ini dengan menentukan package name atau memberikan sebuah permission ke pesan broadcast.
- Jangan memulai activity dari sebuah broadcast yang diterima. Tampilkan saja pesan notifikasi Poin-poin berikut adalah beberapa perubahan pada sistem broadcast receiver yang relevan. Untuk setiap versi beberapa limitasi diberikan dan perilakunya juga berubah. Reviu kembali limitasi ini saat akan membuat sebuah broadcast receiver.

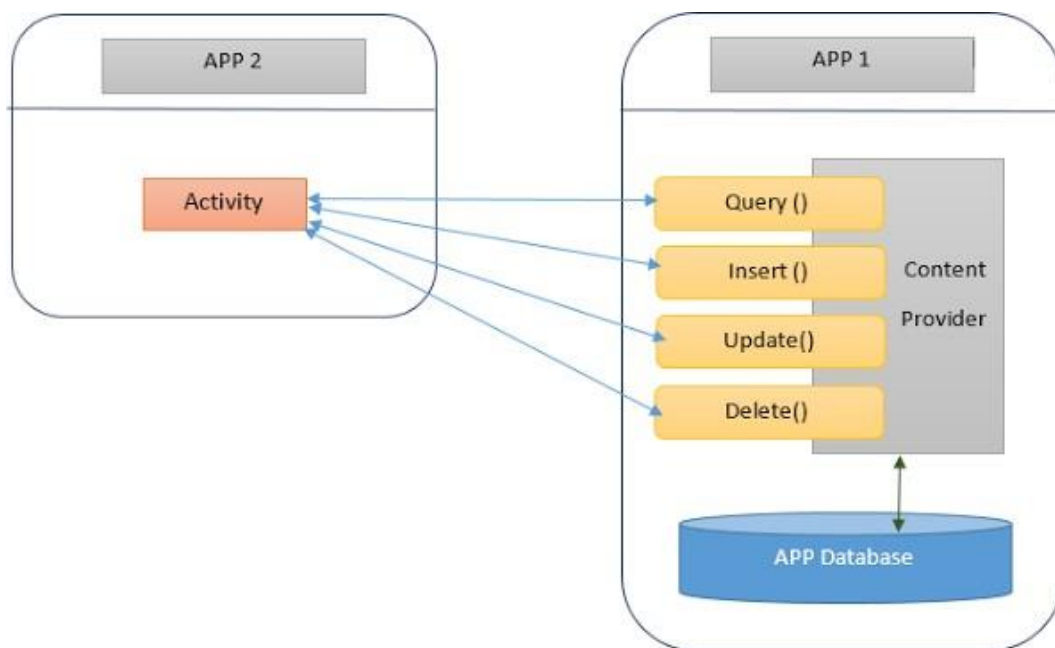
- **7.0 ke atas (API level 24)** — Dua broadcast yang berasal dari sistem ditiadakan yaitu [Action New Picture](#) dan [Action New Video](#) (tapi bisa kembali lagi di Android O dengan pendaftaran receiver lewat Java/Kotlin)
- **8.0 ke atas (API level 26)** — Sebagian besar implicit broadcast perlu diregistrasikan secara dinamis (bukan melalui manifest).
- **9.0 ke atas (API level 28)** — Informasi yang diterima dari broadcast Wi-Fi dari sistem dikurangi dan adanya action bernama [Network State Changed Action](#).

Perubahan di Android Q adalah perubahan yang paling penting untuk diketahui. Perubahan ini dibuat untuk mengatasi masalah performa perangkat, konsumsi baterai yang boros sehingga bisa mengganggu experience pengguna. Masalah tersebut terjadi karena banyak aplikasi (bahkan yang sedang tidak aktif) menunggu event yang terjadi di sistem sehingga ketika terjadi perubahan, sistem akan memakan sumber daya yang besar untuk mengirimkannya. Bayangkan setiap aplikasi yang menunggu suatu event tiba-tiba langsung aktif untuk melakukan sesuatu dalam rangka merespon broadcast yang diterima. Apalagi jika yang ditunggu adalah perubahan seperti status koneksi Wi-Fi yang amat sering terjadi.

4. Content Providers

Content providers menyediakan cara yang fleksibel untuk membuat data yang tersedia di seluruh aplikasi. Misalkan ingin membuat semua jenis data dalam aplikasi (misalnya membuat daftar pengguna dan menyimpannya pada setiap lokasi penyimpanan, mungkin dalam data base, sistem file atau dalam ruang penyimpanan online. Kemudian melalui content provider, aplikasi lain dapat melakukan query, akses atau bahkan memodifikasi data yang telah dibuat. Dalam cara yang sama pengguna dapat mengakses data yang digunakan utilitas lain, menggunakan content provider , sebagai contoh database kontak. Content Provider database kontak memungkinkan aplikasi lain untuk query, membaca, memodifikasi, dan menulis info kontak.

Semua content provider diimplementasikan sebagai subclass dari kelas `ContentProvider` yang didefinisikan dalam Android SDK. Komponen content provider menyuplai data dari satu aplikasi ke aplikasi lainnya pada saat diminta datanya. Request ini ditangani oleh metode dari kelas `Content Resolver`. Datanya mungkin dapat disimpan di file system, di database atau di suatu tempat secara keseluruhan.



C. Penggunaan Intent untuk Mengaktifkan Komponen dalam Aplikasi Android

Tiga dari empat tipe komponen, yaitu activities, services, dan broadcast receiver diaktifkan oleh pesan asinkron yang disebut *intent*. Tidak seperti activities, services, dan broadcast receiver, Content Providers tidak diaktifkan oleh intent. Melainkan, diaktifkan saat ditargetkan oleh permintaan dari `ContentResolver`, yang menangani semua transaksi langsung dengan Content Provider sehingga

komponen yang melakukan transaksi dengan Content Provider tidak perlu dan sebagai gantinya memanggil metode pada objek ContentResolver. Ini membuat layer abstraksi antara Content provider dan komponen yang meminta informasi (demi keamanan).

Intent mengikat antar masing-masing komponen di runtime. Intent dapat dianggap sebagai messenger yang meminta tindakan dari komponen lain, entah komponen milik aplikasi atau komponen lainnya. **Intent** adalah mekanisme untuk melakukan *Action* pada komponen dalam satu aplikasi yang sama maupun aplikasi lain. Pemanfaatannya sebagai berikut :

- Untuk berpindah halaman dari satu *Activity* ke *Activity* lain dengan atau tanpa data
- Untuk menjalankan *background Service* misal dibutuhkan sebuah *proses service* untuk mengambil lokasi pengguna, *download file* atau sinkronisasi ke *server*
- Untuk menyampaikan sebuah objek dari komponen *Broadcast* misal jika ingin mengetahui jika *device Android* sudah selesai *booting* setelah diaktifkan



Explicit dan Implicit Intent

- *Explicit Intent*

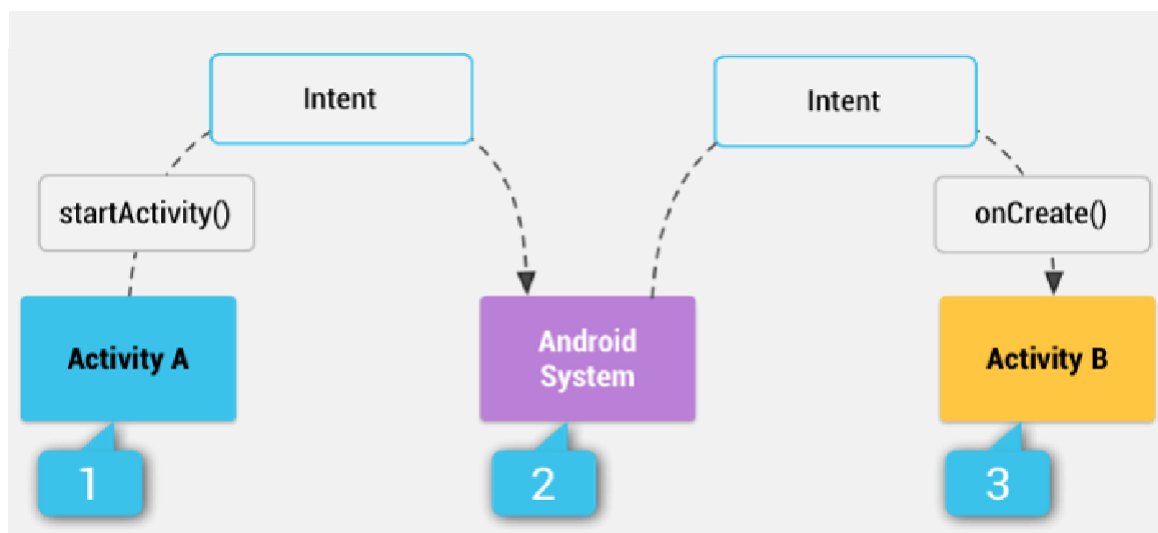
Untuk mengaktifkan komponen-komponen dalam satu aplikasi yang sama misal : Berpindah *Activity* atau mengaktifkan *service* untuk *mendownload file* secara *background*

- *Implicit Intent*

Untuk mengaktifkan komponen dari aplikasi lain. Misal : mengaktifkan *dial phone* pada aplikasi Telp, mengaktifkan *driving direction* pada *Google Maps* atau mengirimkan pesan via *Gmail*, sms atau aplikasi lainnya.

Ilustrasi bagaimana Intent mengaktifkan *Activity* pada aplikasi lain.

1. *Activity A* membuat *Intent* untuk melakukan sebuah *Action* dengan data *object* yang ada.
2. Sistem Android akan mencari aplikasi yang cocok berdasarkan *Intent filter*
3. *Activity* pada aplikasi yang cocok akan ditampilkan.



LEMBAR KERJA

- 1. Lakukan Analisa Lapisan arsitektur sistem operasi android pada smartphone Anda. Buatlah Analisa tersebut dalam bentuk table, bagan atau bentuk lain sesuai kretaivitas tim Anda.**
- 2. Lakukan Analisa Komponen Aplikasi android pada smartphone Anda. Buatlah Analisa tersebut dalam bentuk table, bagan atau bentuk lain sesuai kretaivitas tim Anda.**
- 3. Lakukan Analisa penggunaan inten untuk mengaktifkan komponen dalam aplikasi android pada smartphone Anda. Buatlah Analisa tersebut dalam bentuk table, bagan atau bentuk lain sesuai kretaivitas tim Anda.**

BAB III INTEGRATED DEVELOPMENT ENVIRONMENT ANDROID STUDIO

TUJUAN PEMBELAJARAN

Setelah berdiskusi kelompok dan praktek peserta mampu:

1. Melakukan Instalasi Android Studio
2. Mengaplikasikan pengaturan layout dengan benar
3. Mengaplikan penggunaan user interface dengan benar

1.1 Pengenalan Android Studio

Android Studio adalah Integrated Development Environment (IDE) resmi untuk pengembangan aplikasi Android. Dikembangkan oleh Google dan berbasis pada IntelliJ IDEA, Android Studio menyediakan berbagai alat dan fitur untuk membantu pengembang membangun aplikasi Android yang berkualitas tinggi. Berikut adalah pengenalan dasar tentang Android Studio dan cara menggunakannya:

Fitur Utama Android Studio

1. Editor Layout Visual:

- Android Studio memiliki editor layout visual yang kuat, memungkinkan Anda untuk mendesain antarmuka pengguna aplikasi Anda dengan menyeret dan menjatuhkan komponen UI.

2. Lint Tools:

- Lint tools membantu menemukan dan memperbaiki masalah dalam file XML, sumber daya, dan file Java atau Kotlin.

3. Instant Run:

- Instant Run memungkinkan Anda untuk melihat perubahan kode langsung tanpa harus membangun ulang aplikasi Anda.

4. Gradle-based Build System:

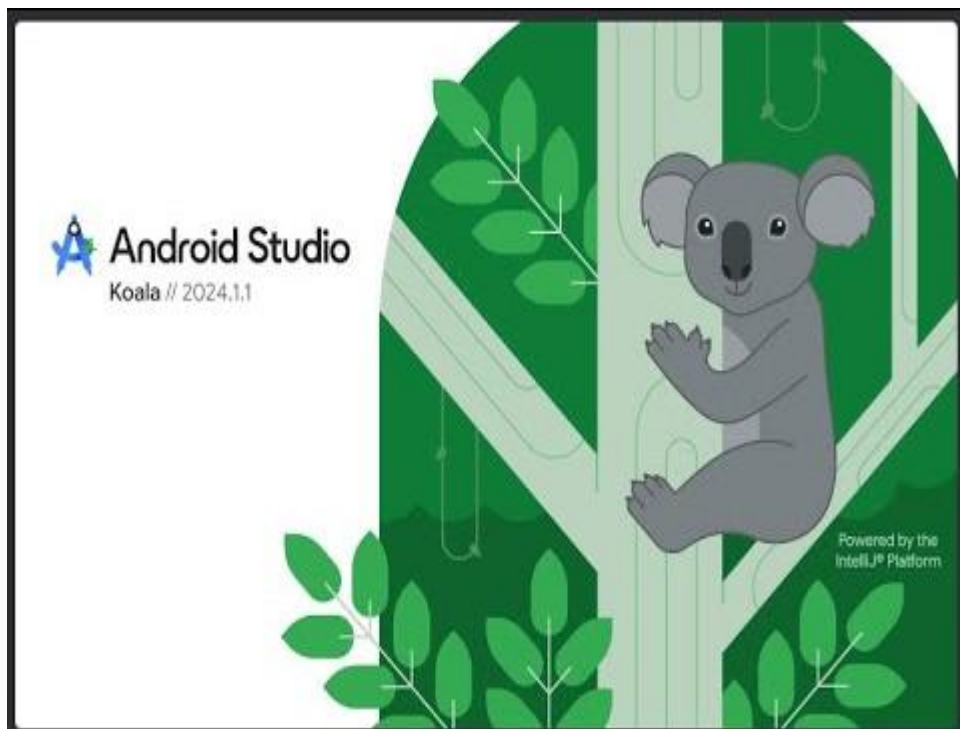
- Android Studio menggunakan Gradle sebagai sistem build, memungkinkan Anda untuk mengelola dependensi dan build varian dengan mudah.

5. **Emulator Android:**

- Emulator Android memungkinkan Anda menguji aplikasi di berbagai perangkat Android virtual dengan cepat.

6. **IntelliJ-based Code Editor:**

- Menyediakan fitur-fitur seperti auto-completion, refactoring, dan analisis kode.



Berbicara tentang pemrograman tentunya tidak lepas dari Integrated Development Environment (IDE) yang bisa dipakai oleh para Developer. Pada 2014 Google mengeluarkan IDE yang bernama Android Studio yang berbasikan dari IntelliJ IDEA. Dengan menggunakan Android Studio para developer dapat membuat aplikasi dari nol hingga publish ke dalam store. Android Studio juga mempunyai beberapa fitur built-in yang sangat membantu para developer untuk memaksimalkan proses pembuatan aplikasi seperti Gradle, Code Completion, dan terintegrasi dengan beberapa services dari Google seperti Firebase. Salah satu Bahasa yang bisa digunakan untuk development Android adalah Java. Selain Java ada beberapa Bahasa lain yang bisa digunakan seperti C/C++, Go, dan pada May 2017 Google resmi

support Kotlin. Pada kegiatan pembelajaran ini akan menggunakan Kotlin sebagai Bahasa pemrograman. program.

Kotlin adalah bahasa pemrograman yang modern, yang dikembangkan oleh JetBrains. Kotlin dirancang untuk sepenuhnya interoperable dengan Java dan untuk meningkatkan pengalaman pemrograman dengan mengatasi beberapa kelemahan Java. Kotlin kini menjadi bahasa resmi untuk pengembangan Android dan didukung secara penuh oleh Google.

Keunggulan Kotlin

1. Interoperabilitas dengan Java:

- Kotlin dapat bekerja dengan baik bersama kode Java. Anda bisa memanggil kode Java dari Kotlin dan sebaliknya tanpa masalah.

2. Sintaks yang Lebih Bersih dan Ringkas:

- Kotlin dirancang untuk lebih ringkas dan ekspresif dibandingkan Java. Ini berarti lebih sedikit kode yang harus ditulis untuk mencapai hal yang sama.

3. Null Safety:

- Kotlin memiliki fitur null safety bawaan yang membantu menghindari NullPointerException, salah satu kesalahan paling umum dalam pemrograman Java.

4. Fungsi Ekstensi:

- Anda bisa menambahkan fungsionalitas baru ke kelas-kelas yang ada tanpa harus mewarisinya. Ini membuat kode lebih modular dan mudah dipahami.

5. Coroutines:

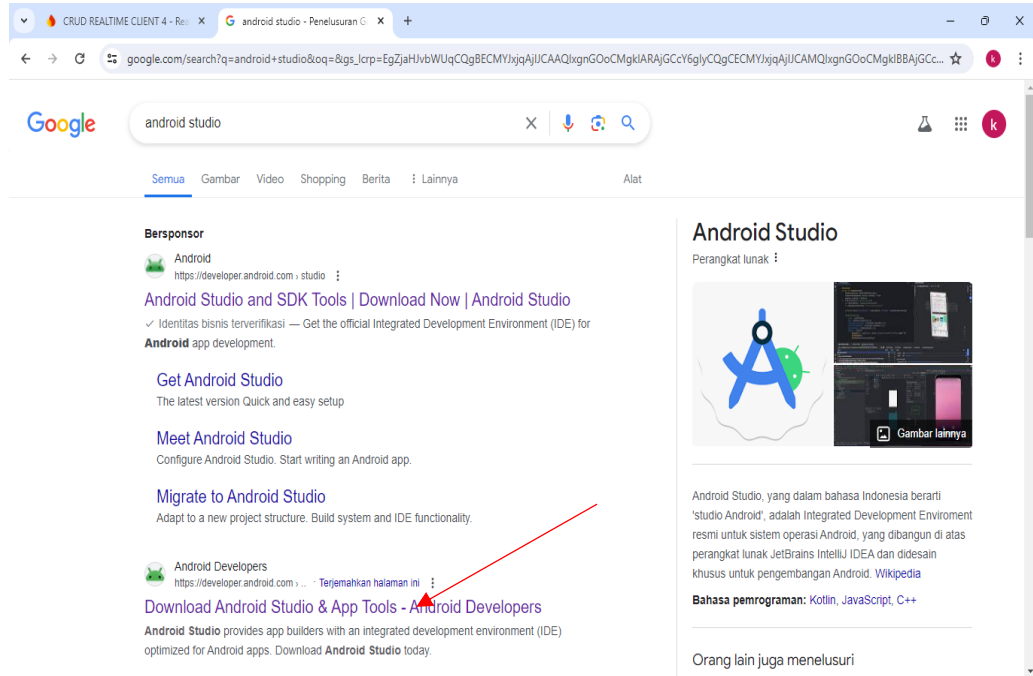
- Kotlin mendukung coroutines, yang membuat penanganan operasi asynchronous lebih sederhana dan lebih mudah dipahami.

Silakan lakukan tahapan instalasi Android Studio dengan mengikuti langkah-langkah pada LEMBAR KERJA INSTALASI ANDROID STUDIO berikut ini.

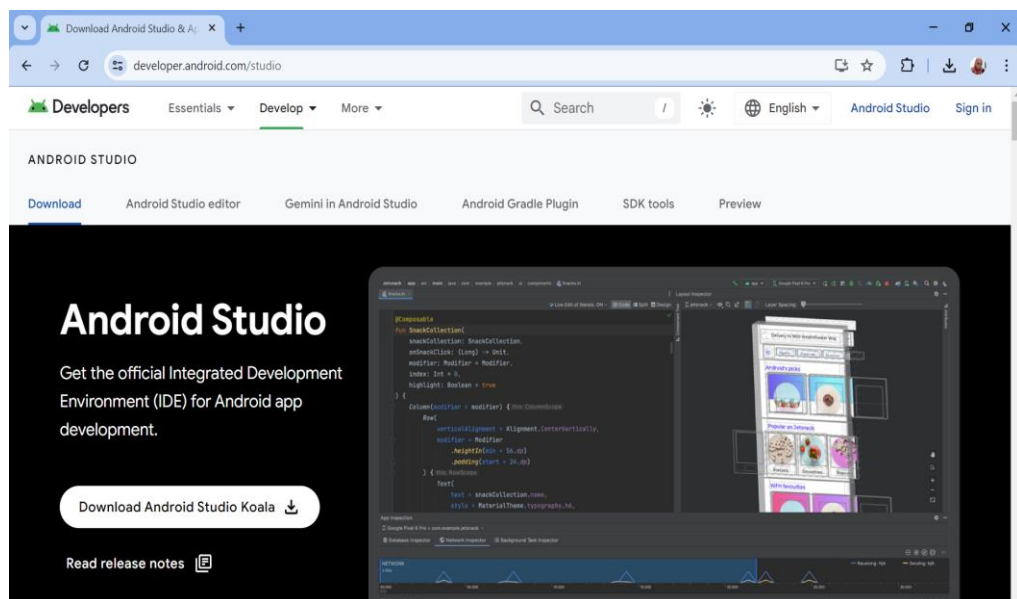
LEMBAR KERJA INSTALASI ANDROID STUDIO

Berikut ini cara instalasi android studio

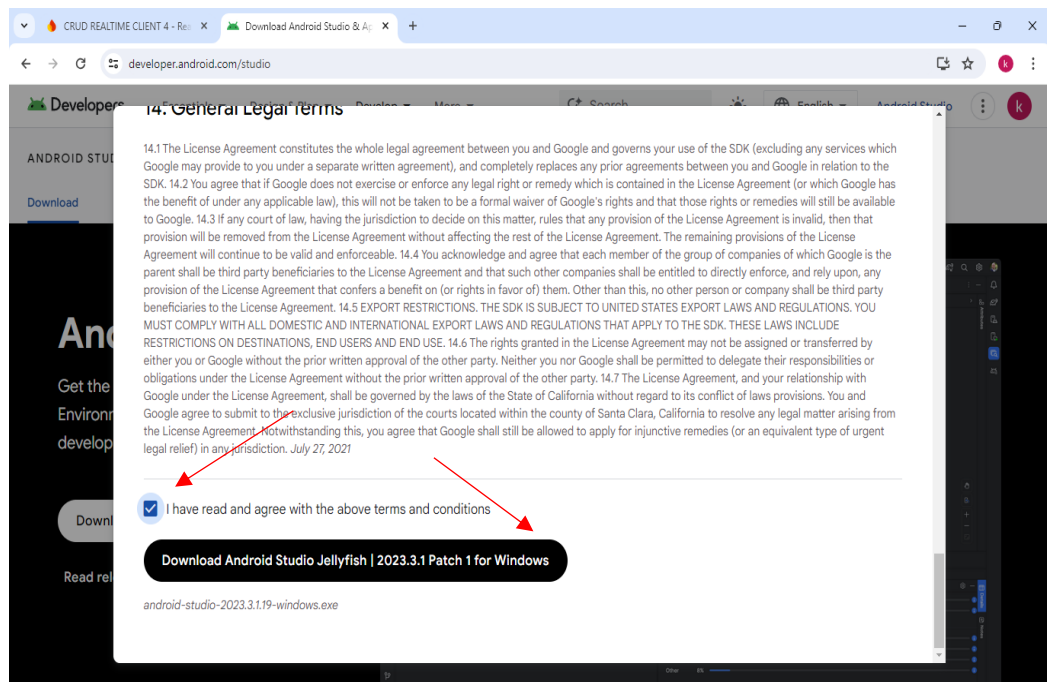
1. Cari Android Studio di google chrome dan klik seperti di gambar



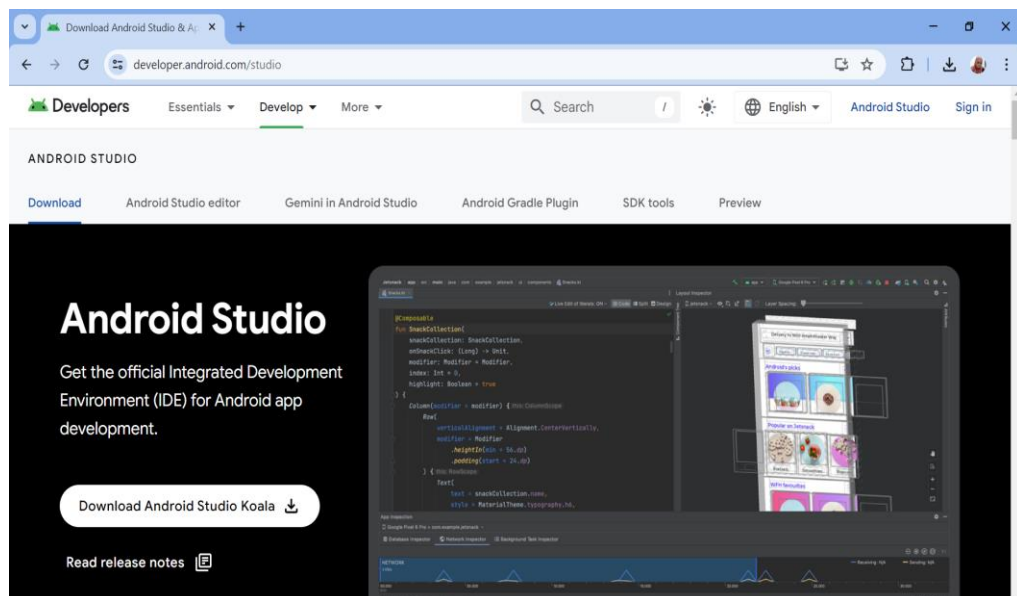
2. Klik download seperti gambar dibawah



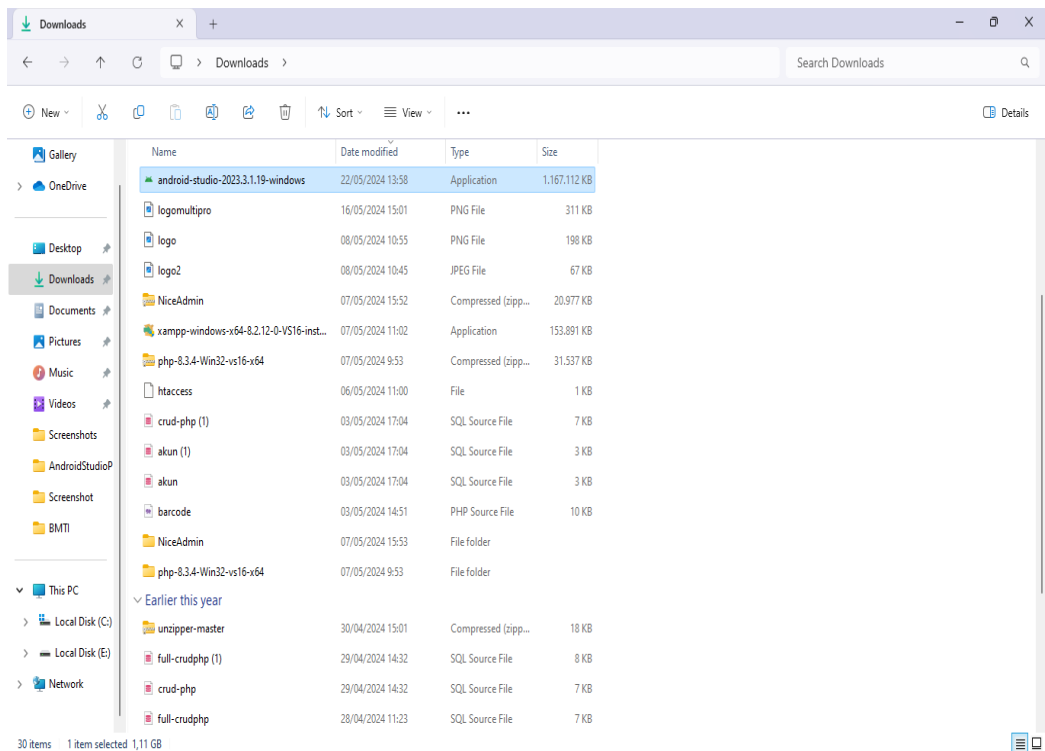
3. Scroll sampai bawah kemudian centang dan klik download



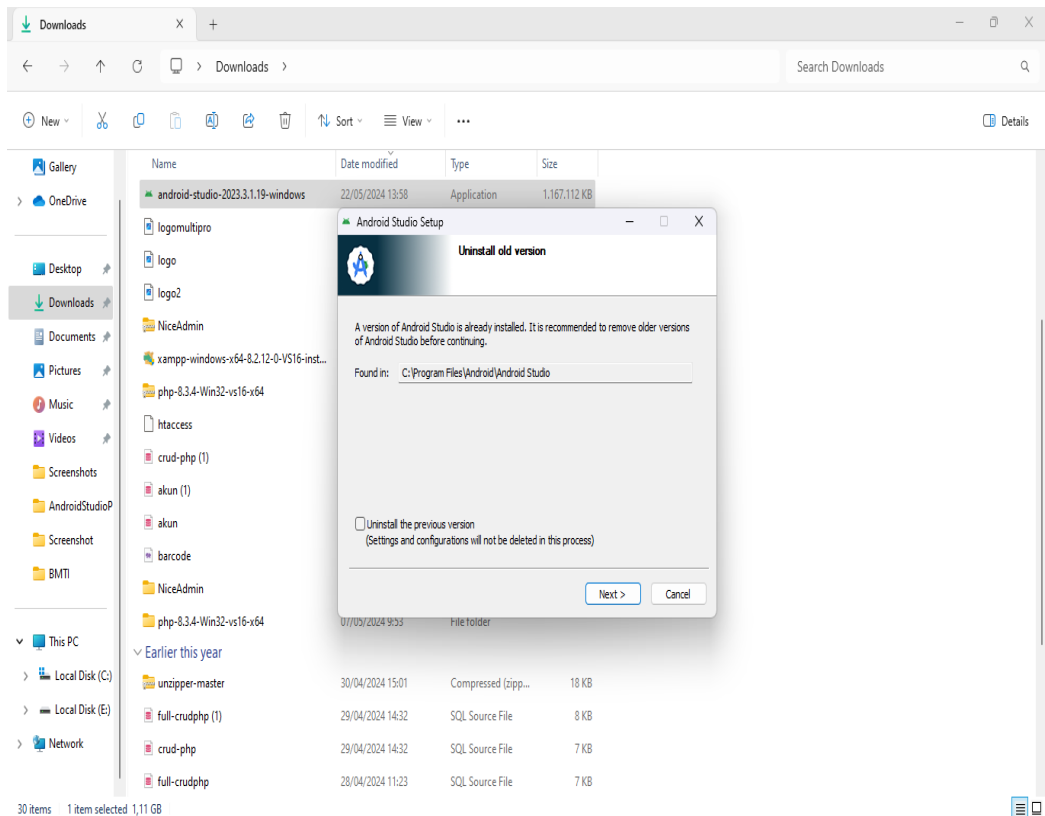
4. Tunggu hingga download selesai



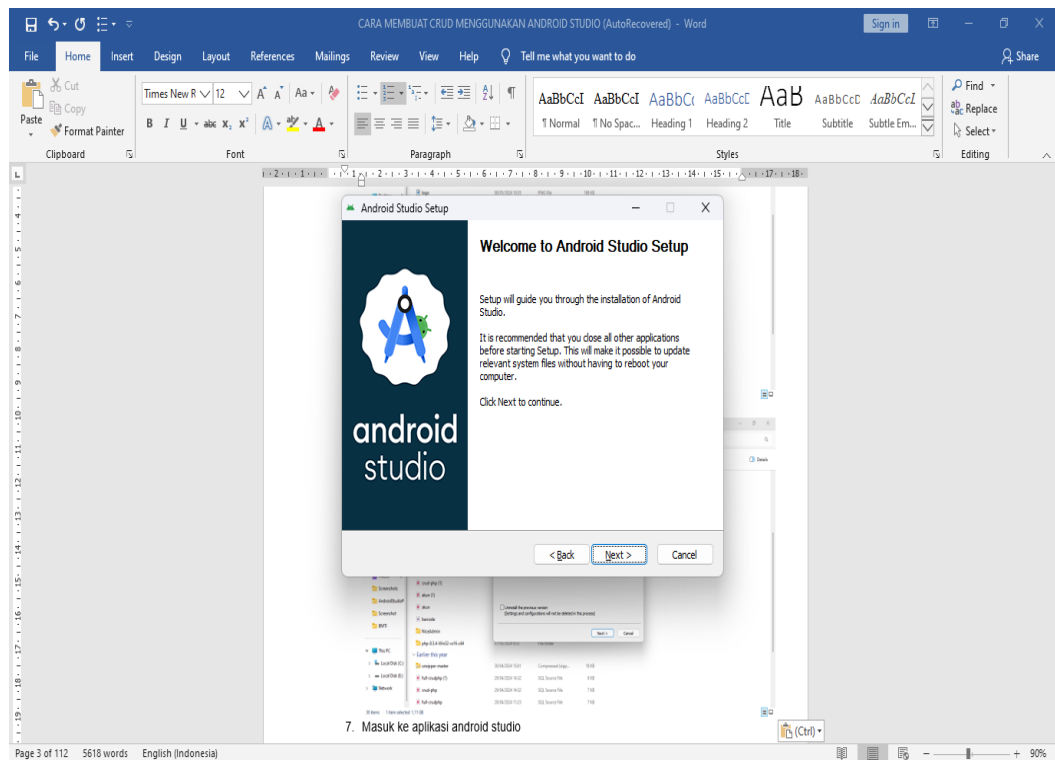
5. Jika sudah terdownload buka dengan cara meng klik dua kali



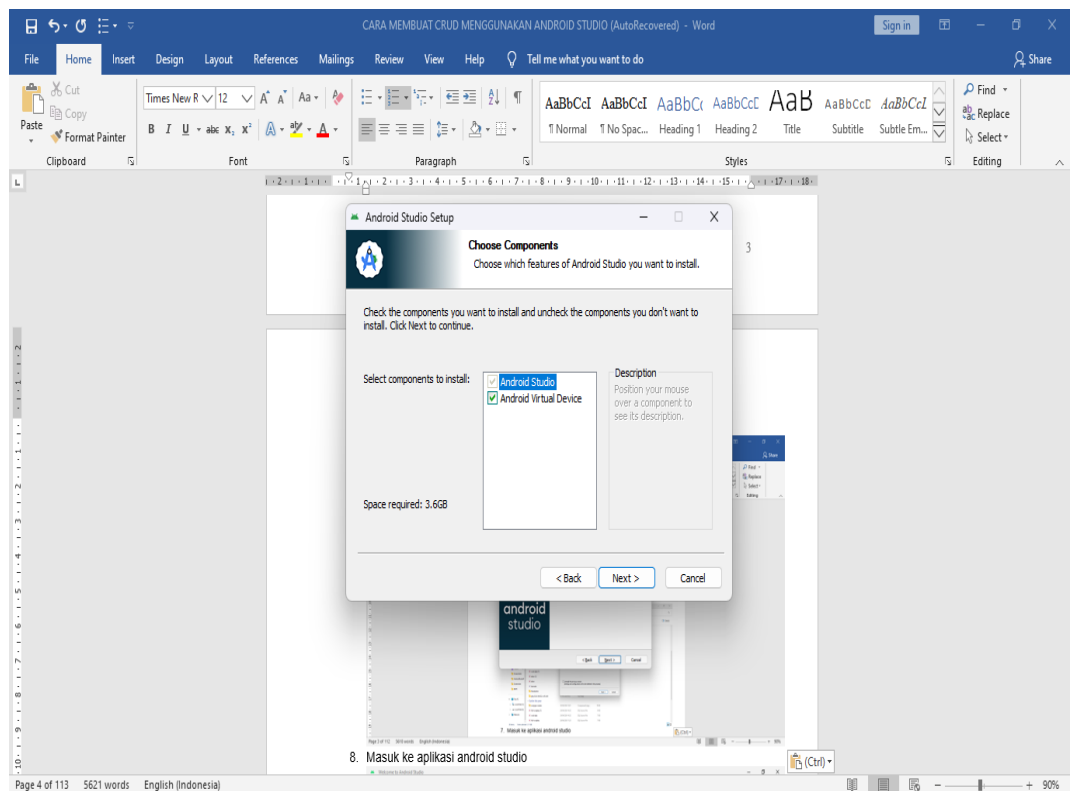
6. Pilih penyimpanan untuk menyimpan android studio lalu Klik Next



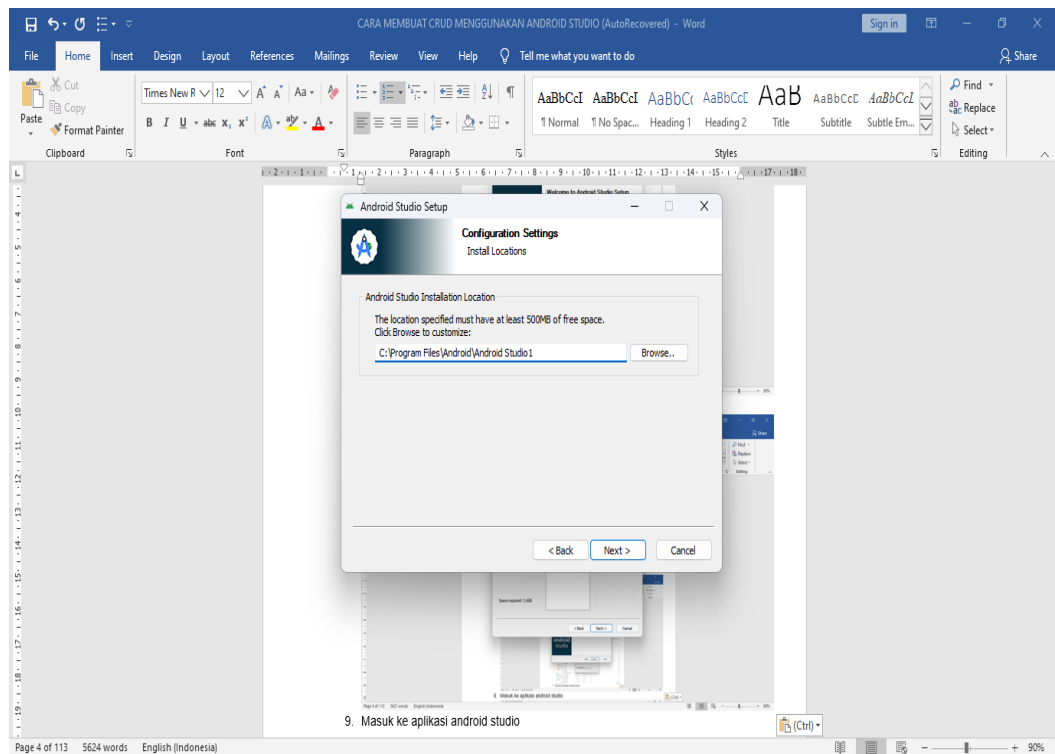
7. Klik Next



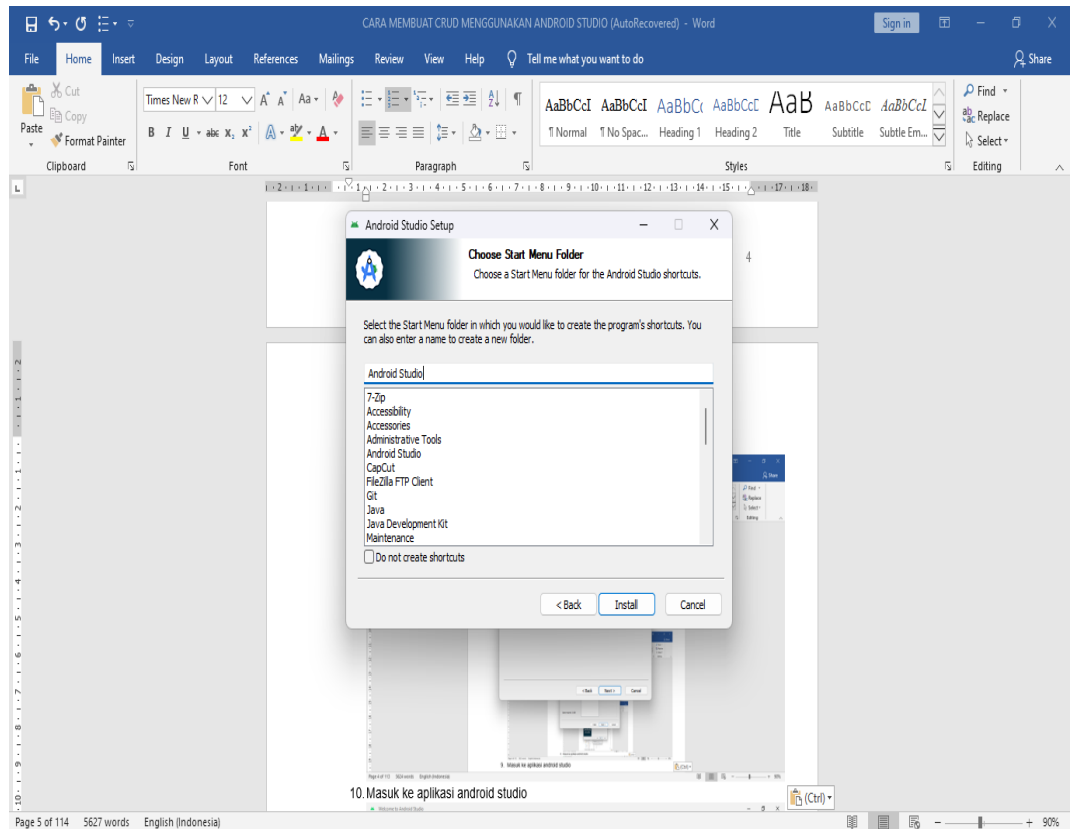
8. Pilih component kemudian Klik Next



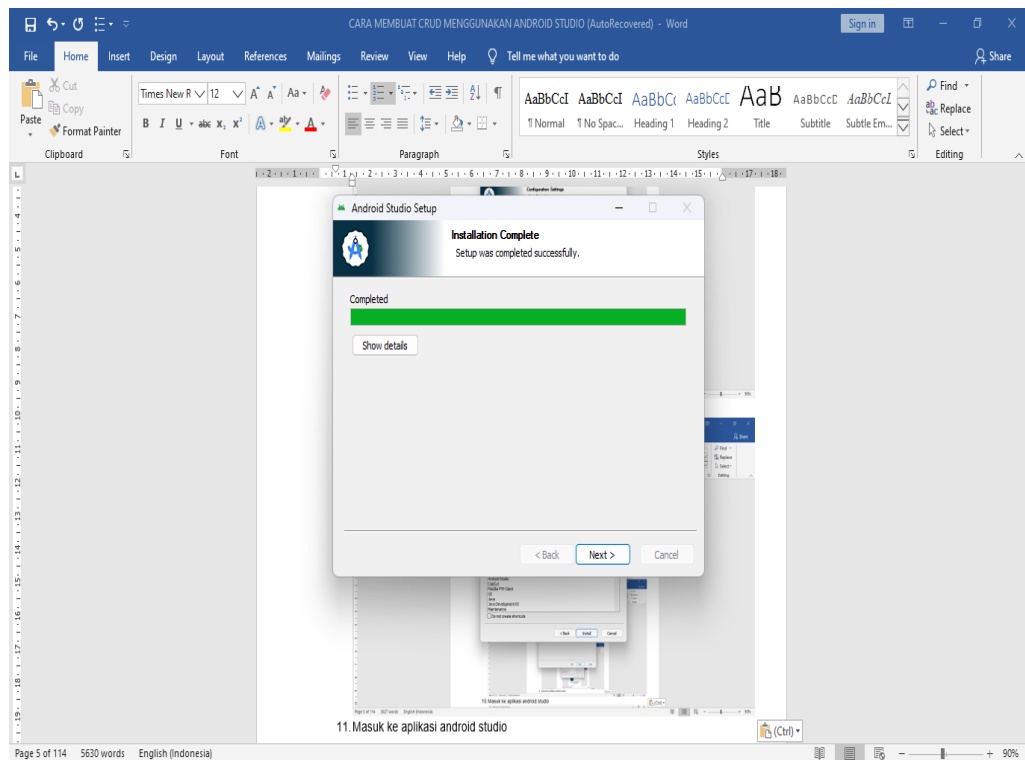
9. Pilih penyimpanan untuk menyimpan android studio lalu Klik Next



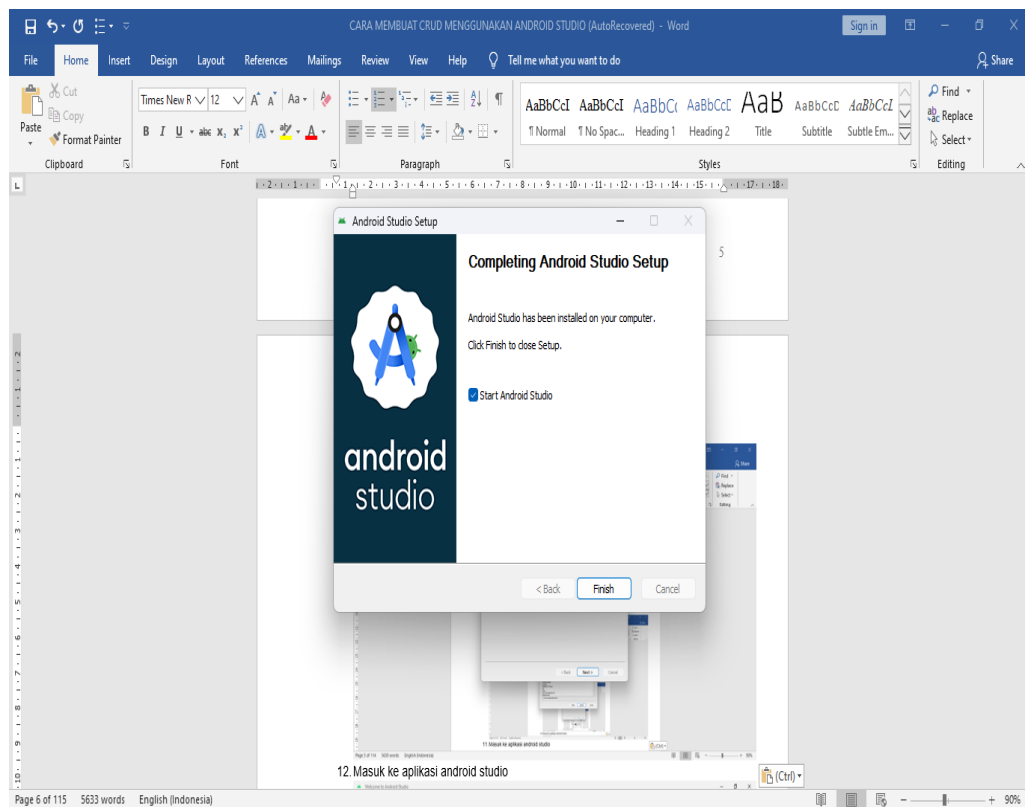
10. Pilih atau buat folder lalu Klik Install



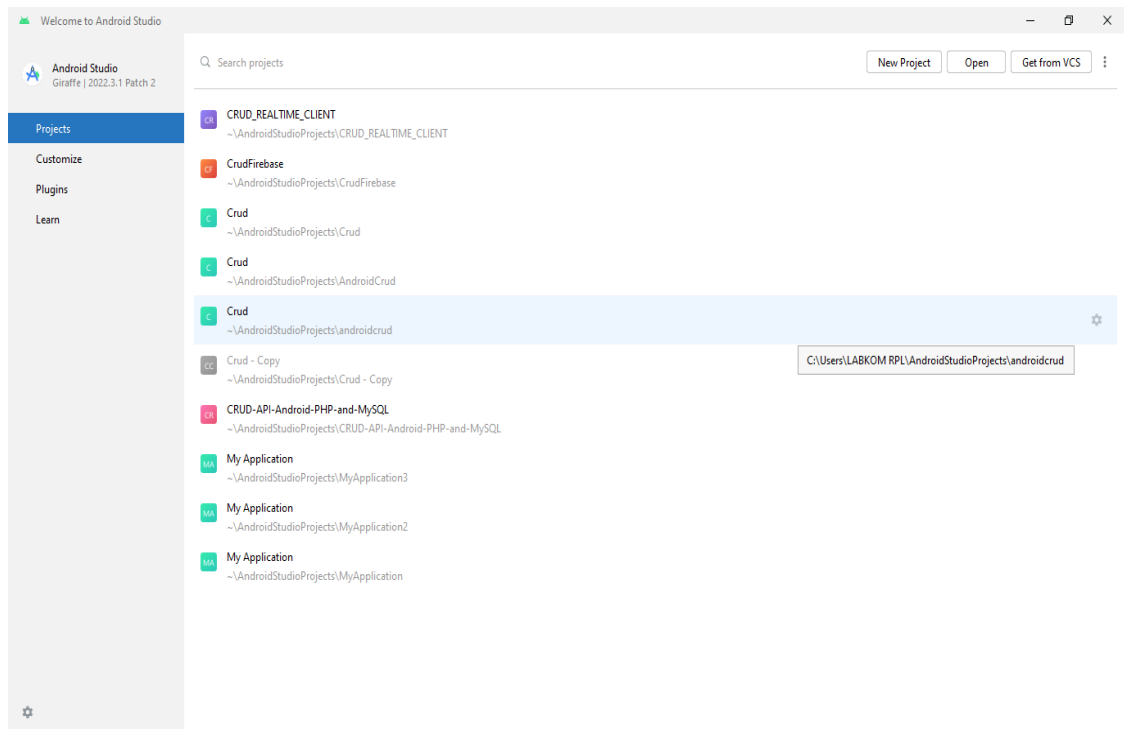
11. Tunggu hingga proses intallasi complete kemudian Klik Next



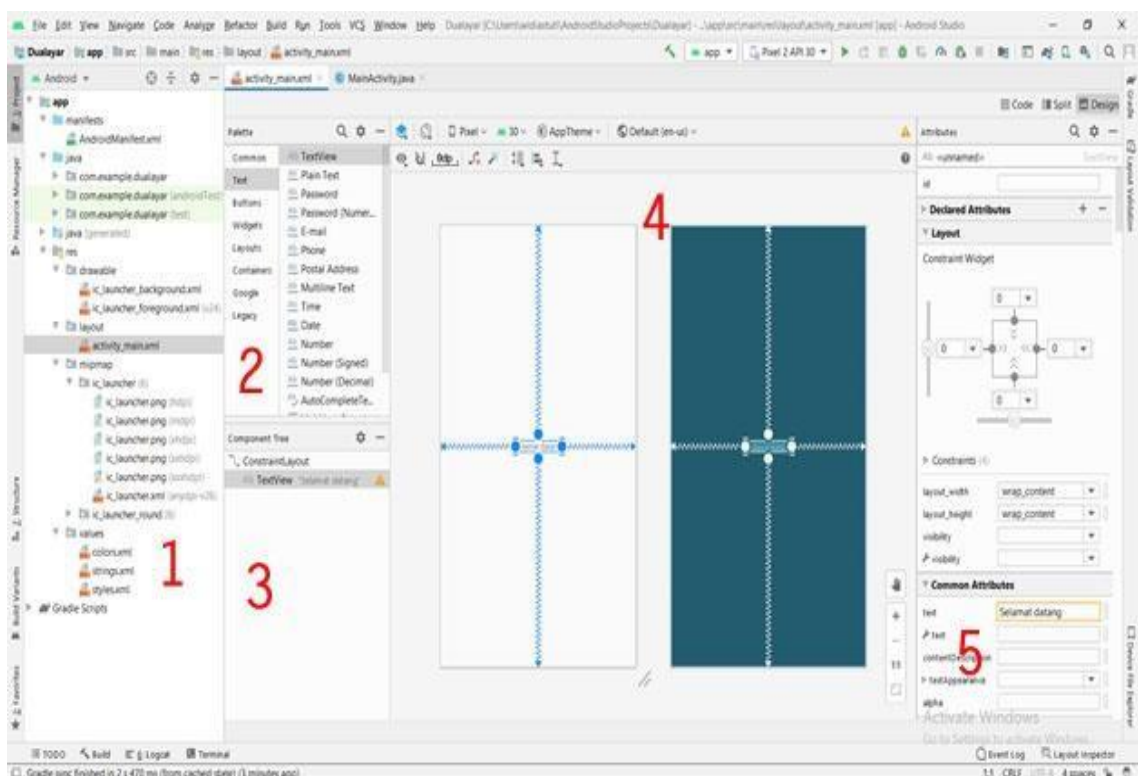
12. Klik Finish



13. Masuk ke aplikasi android studio



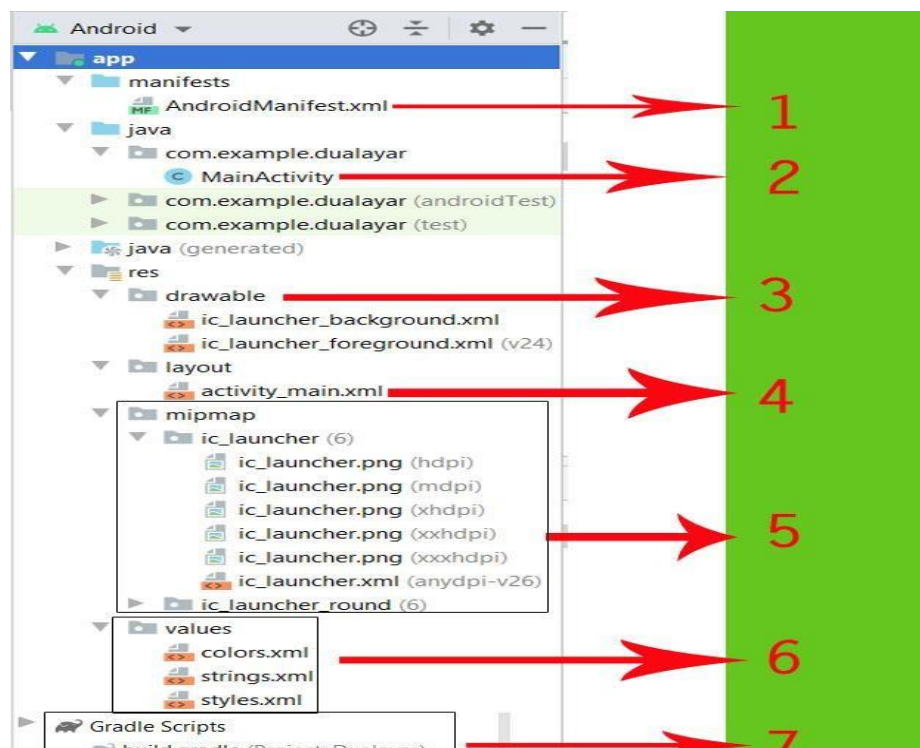
1.2 Struktur Project Android Studio



Android Studio jika sudah terinstall, akan mempunyai lingkungan kerja seperti tertera pada gambar berikut ini.

No	PENJELASAN
1.	Merupakan bagian dari susunan folder project Android Studio
2.	Merupakan bagian dari Palette yang berisi beberapa tools atau component yang bisa kita gunakan untuk memberi tampilan pada aplikasi yang akan kita bangun.
3.	Merupakan <i>Component Tree</i> yang memiliki fungsi menampilkan Layout dan susunan dari komponen yang telah kita gunakan di Bagian 4.
4.	Merupakan bagian bidang desain yang digunakan untuk meletakkan <i>Component</i> yang ingin kita gunakan. Tampilannya seperti tampilan layar pada Mobile Android.
5.	Merupakan bagian <i>Properties</i> , bagian ini akan menampilkan semua informasi dari <i>Component</i> yang kita pilih seperti nama id, ukuran/Size, Font, background, color, dan lain sebagainya.

Ketika membuat sebuah project pada Android Studio, maka akan terlihat bagian Struktur Project seperti berikut ini.



Berikut ini merupakan penjelasan dari struktur project tersebut.

NO	PENJELASAN
1	Pada bagian pertama terdapat folder yang bernama Manifest, di dalam folder Manifest tersebut terdapat file AndroidManifest.xml. File AndroidManifest.xml berisi informasi tentang package (paket) pada project yang akan dibangun. Jadi file ini memuat komponen aplikasi seperti Activity, Services, Content, dan lain sebagainya. Jika ingin membuat <i>Activity</i> baru maka akan terdapat di file ini, jika tidak maka periksa kembali, dan jika ingin membuat aplikasi yang membutuhkan koneksi Internet, diharuskan mengisi User Permission di File ini, mengganti judul, icon, dan lain sebagainya.
2	Pada bagian kedua terdapat folder Java yang di dalamnya terdapat folder bernama seperti domain yang diisikan saat pertama kali membuat aplikasi, pada contoh ini dibuat dengan nama (com.example.dualayar), di dalam folder tersebut terdapat file MainActivity.java, pada bagian file ini

NO	PENJELASAN
	akan memuat program Java untuk aplikasi yang akan dibuat.
3	Pada bagian ketiga ini terdapat folder drawable , folder drawable ini merupakan bagian dari folder resource . Folder drawable ini digunakan untuk menyimpan file gambar dengan format .jpg atau .png yang akan digunakan pada aplikasi yang akan dibangun
4	Pada bagian keempat ini terdapat file bernama activity_main.xml, File activity_main.xml ini terdapat di folder layout yang merupakan bagian dari folder resource. Pada file ini akan dibuat tampilan User Interface untuk aplikasi yang akan dibangun.
5	Pada bagian kelima terdapat beberapa icon launcher aplikasi yang bernama ic_launcher . Icon ini terdapat pada folder mipmap yang merupakan bagian dari folder resource .
6	Pada bagian keenam ini terdapat folder values yang berisi beberapa file antara lain: • <i>String.xml</i> yang merupakan file berisi <i>resource</i> data-data text yang akan digunakan sebagai komponen aplikasi. • <i>Style.xml</i> yang merupakan file berisi <i>resource</i> tentang tema seperti Toolbar, yang akan kita gunakan untuk komponen aplikasi

NO	PENJELASAN
	nantinya. • <i>Color.xml</i> yang merupakan file berisi Resource warna-warna dari file <i>Style.x</i>
7	Pada bagian ketujuh terdapat Gradle. Gradle ini merupakan tools untuk build yang digunakan pada Android Studio, untuk Compile, Run project Aplikasi yang akan dibangun. Jika pernah menggunakan Eclipse, maka tidak akan menemukan build tools Gradle, karena Eclipse menggunakan build tools Ant.

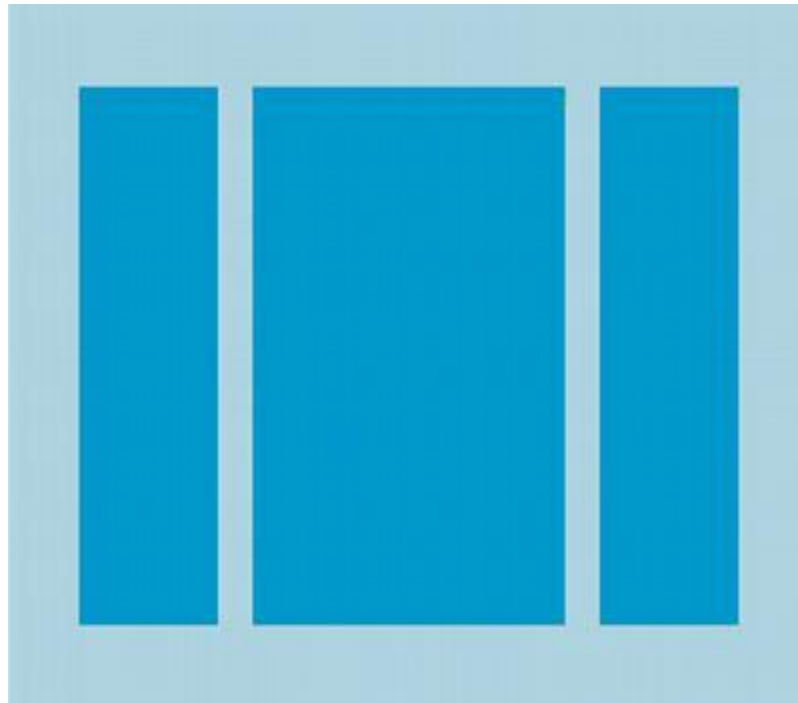
BAB IV PEMBUATAN RESPONSIVE LAYOUT

A. Jenis dan Fungsi Layout

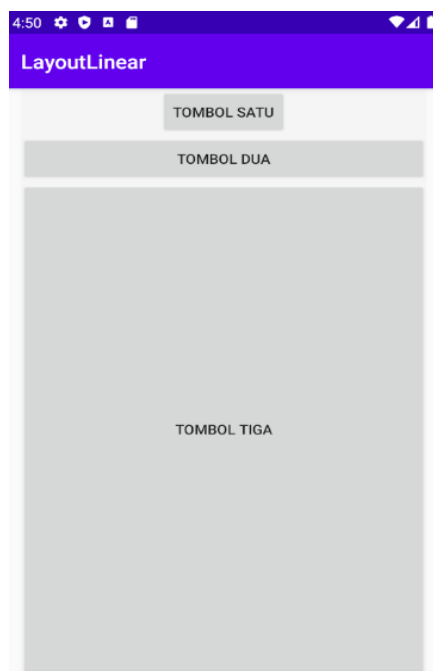
Layout merupakan suatu tampilan tata letak di Android untuk mengatur penempatan teks, gambar, ataupun komponen lainnya sehingga tampilan pada aplikasi yang dibuat terlihat rapih dan nyaman untuk dilihat oleh pengguna. Jenis dan fungsi Layout adalah sebagai berikut:

1. Linear Layout

Linear Layout adalah layout yang menyejajarkan semua *child view*-nya dalam satu arah, secara vertikal atau horizontal. Penggunaan linear layout dapat menetapkan arah layout menggunakan atribut **android:orientation**.



Jika menggunakan android orientation vertikal maka pengaturan layout akan berurutan dari atas ke bawah secara vertical. Jika menggunakan android orientation horizontal maka pengaturan layout akan berurutan dari kiri ke kanan secara horizontal. pengisi). Linear Layout akan mengikuti *margin* dan *gravity* sejajar kanan, tengah, atau kiri. Berikut ini contoh desain aplikasi android menggunakan linear layout.



2. Relative Layout

Relative Layout merupakan pengaturan layout yang menempatkan widget-widget didalamnya seperti layer, sehingga sebuah widget dapat berada di atas atau di bawah widget lainnya. Jika dibandingkan dengan linear layout maka Relative Layout merupakan layout yang penataannya lebih bebas karena penempatan widget tidak perlu berurutan dari atas ke bawah atau dari kiri ke kanan seperti pada Linear Layout, melainkan widget dapat ditempatkan dimana saja dengan mengacu pada widget lain.



Jadi dapat diartikan bahwa **Relative Layout** dapat membuat desain tampilan aplikasi dengan tata letak objek atau komponen secara bebas tanpa aturan sesuai orientasi (horizontal atau vertical) seperti yang diterapkan pada **Linear Layout**. Untuk mengatur posisi elemen User Interface, misalnya Button, cukup drag pada posisi yang diinginkan, tapi jika ingin menggunakan kode XML, ada beberapa atribut yang bisa digunakan pada elemen atau komponen UI didalam Relative Layout yaitu:

- **android:layout_centerInParent:** jika kita set menjadi "**true**", elemen UI akan berada di tengah relatif terhadap lebar dan tinggi Layout.
- **android:layout_centerHorizontal:** jika kita set menjadi "**true**", elemen UI akan berada di tengah relatif terhadap lebar Layout.
- **android:layout_centerVertical:** jika kita set menjadi "**true**", elemen UI akan berada di tengah relatif terhadap tinggi Layout.
- **android:layout_alignParentBottom:** jika kita set menjadi "**true**", elemen UI diletakan rata sisi bawah layout.
- **android:layout_alignParentTop:** jika kita set menjadi "**true**", elemen UI diletakan rata sisi atas layout.
- **android:layout_alignParentLeft:** jika kita set menjadi "**true**", elemen UI diletakan rata sisi kiri layout.
- **android:layout_alignParentRight:** jika kita set menjadi "**true**", elemen UI diletakan rata sisi kanan layout.

Ada beberapa atribut yang bisa dikombinasikan, misalnya `android:layout_centerHorizontal` dengan `android:layout_alignParentBottom`. Jika kedua atribut tersebut di set menjadi "**true**", komponen UI akan berada di tengah bagian bawah layout. Selain itu ada juga beberapa atribut yang tidak bisa dikombinasikan atau bisa disebut **mutual exclusive**, yaitu hanya bisa dipilih satu misalnya `android:layout_centerVertical` dengan `android:layout_alignParentBottom`.

Pengaturan posisi relatif antara elemen UI dengan elemen UI lainnya juga dapat dilakukan, ada beberapa atribut yang dapat digunakan yaitu:

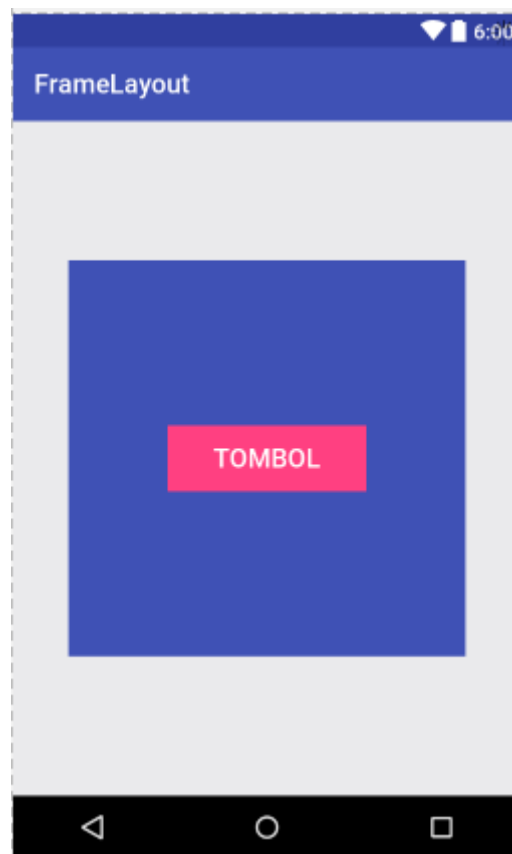
- **android:layout_above**: jika di set menjadi "**true**", elemen UI akan berhimpit atau berada di atas elemen UI yang di picu/dituju.
- **android:layout_below**: jika di set menjadi "**true**", elemen UI akan berhimpit atau berada di bawah elemen UI yang di picu/dituju.
- **android:layout_toLeftOf**: jika di set menjadi "**true**", elemen UI akan berhimpit atau berada di sisi kiri elemen UI yang di picu/dituju.
- **android:layout_toRightOf**: jika di set menjadi "**true**", elemen UI akan berhimpit atau berada di sisi kanan elemen UI yang di picu/dituju.

Berikut ini adalah contoh tampilan User Interface menggunakan Relative Layout.



3. Frame Layout

Frame Layout adalah Layout yang biasanya digunakan untuk membuat objek yang saling bertindihan contohnya yaitu kita membuat button di atas image, seperti terlihat pada gambar berikut ini.



4. Table Layout

Table Layout adalah Layout yang digunakan untuk membangun user interface (tampilan antar muka) aplikasi android dengan berdasarkan baris dan kolom.

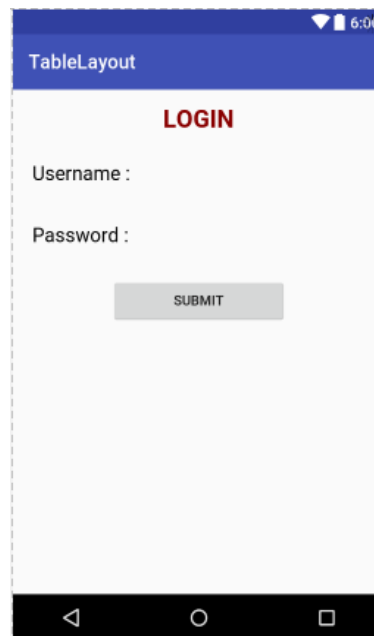
Table Layout terdiri dari:

1. **Row/ baris** pada dasarnya digunakan untuk menyimpan satu jenis record, hanya satu informasi yang dapat disimpan.
2. **Kolom** adalah sub bagian terbagi dari setiap baris dan satu baris dapat menampung beberapa jenis kolom.

Setiap kolom terdiri dari jenis informasi yang berbeda mengenai baris tersebut. Beberapa tag pada Table Layout yaitu:

1. **TableLayout** : Tag pembuka untuk menggunakan TableLayout
2. **TableRow** : Tag untuk membuat Baris

Berikut ini adalah contoh penggunaan Table Layout.



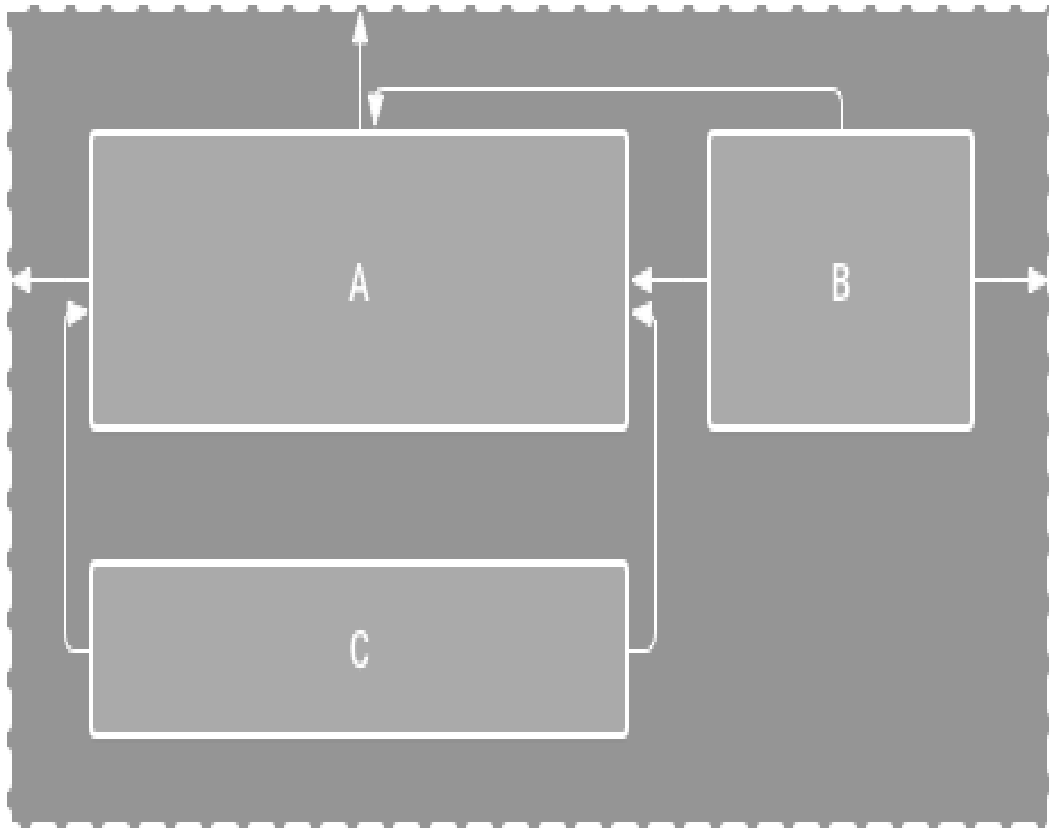
Gambar 1.6 Table Layout

5. Constraint Layout

Sebenarnya **Constraint Layout** mirip dengan **Relative Layout**, karena letak View bergantung pada View lain dalam satu Layout ataupun dengan Parent Layoutnya.

Contohnya di **Relative Layout** kita bisa meletakkan sebuah View di atas, bawah,

atau samping View lain. Kita juga dapat mengatur posisinya berdasarkan Parent Layout menggunakan tag seperti `centerVertical`, `alignParentBottom`. Akan tetapi, **Constraint Layout** jauh lebih fleksibel dan lebih mudah digunakan di **Layout Editor**.



Pada **Constraint Layout**, setiap View memiliki semacam tali (Constraint) yang menarik tiap sisinya, yang mana tali tersebut bisa diatur Elastisitasnya, Marginnya, dan sebagainya. Tali tersebut wajib “diikatkan” kepada anchor point atau suatu titik yang dapat berupa sisi dari Parent Layout, View lain, ataupun titik bantu (helper) yang bisa dibuat sendiri.. Penggunaan Constraint Layout memungkinkan membuat dsain tampilan aplikasi yang responsive sesuai ukuran layar perangkat android dan sesuai orientasi layar portrait atau landscape. Berikut ini contoh penggunaan Constraint Layout untuk membuat tampilan aplikasi yang responsive secara ukuran layar dan orientasi layar.

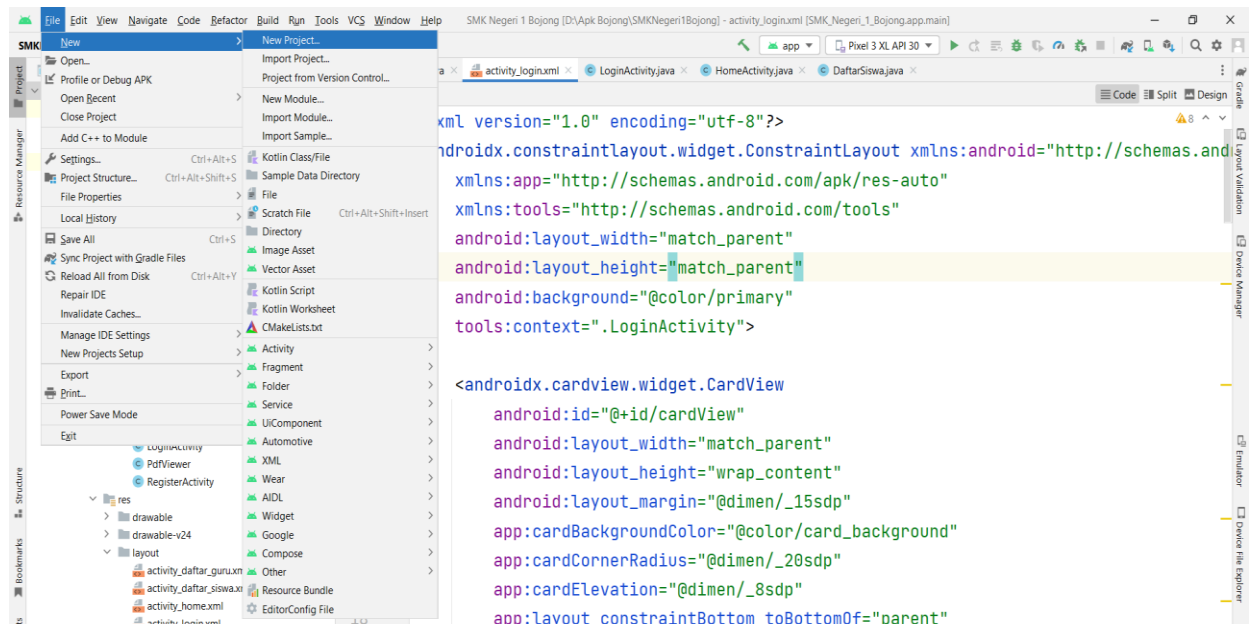


B. LEMBAR KERJA

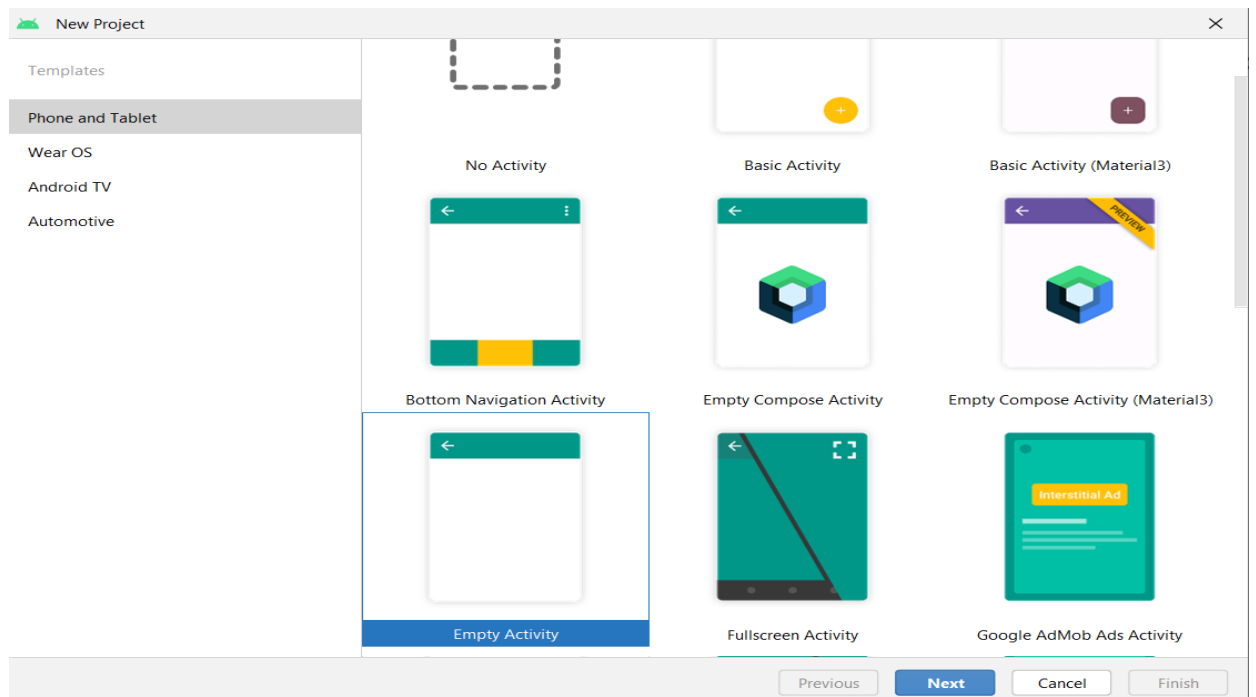
Tampilan aplikasi disebut responsive ketika tampilan aplikasi tersebut rapih dan konsisten pada berbagai ukuran perangkat android misalnya smartphone dengan ukuran 5 inchi, tablet dengan ukuran layar yang lebih besar, baik pada orientasi portrait atau landscape. Responsive Layout pada tampilan aplikasi android dapat dibuat menggunakan Constraint Layout, sehingga pada kegiatan pembelajaran ini akan digunakan pembuatan tampilan aplikasi menggunakan Constraint layout. Dalam Lembar kerja ini, terdapat contoh latihan yang dijelaskan langkah-langkah pengerjaannya secara terperinci dan sudah ada source codenya untuk dipraktekkan. Lembar Kerja ini juga memuat tugas berupa soal untuk dicari pemecahan jawabannya sekaligus dipraktekkan dalam diskusi kelompok pada kegiatan pembelajaran. Semua Lembar kerja baik latihan maupun tugas sebaiknya dipraktekkan, dengan tujuan dapat mengaplikasikan topik materi ini.

➤ LEMBAR KERJA Responsive Layout Berdasarkan Orientasi Layar

1. Buat project baru, dengan klik Menu File, New Project, seperti berikut ini.

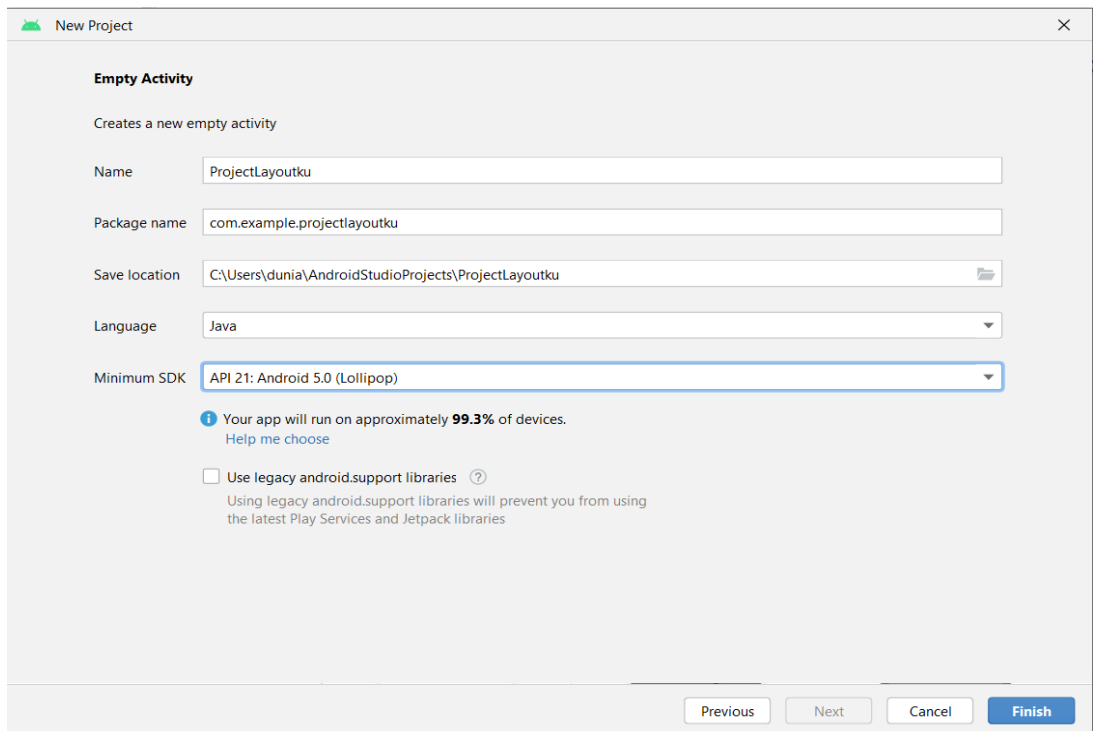


2. Pilih empty activity, seperti berikut ini, lalu klik next.

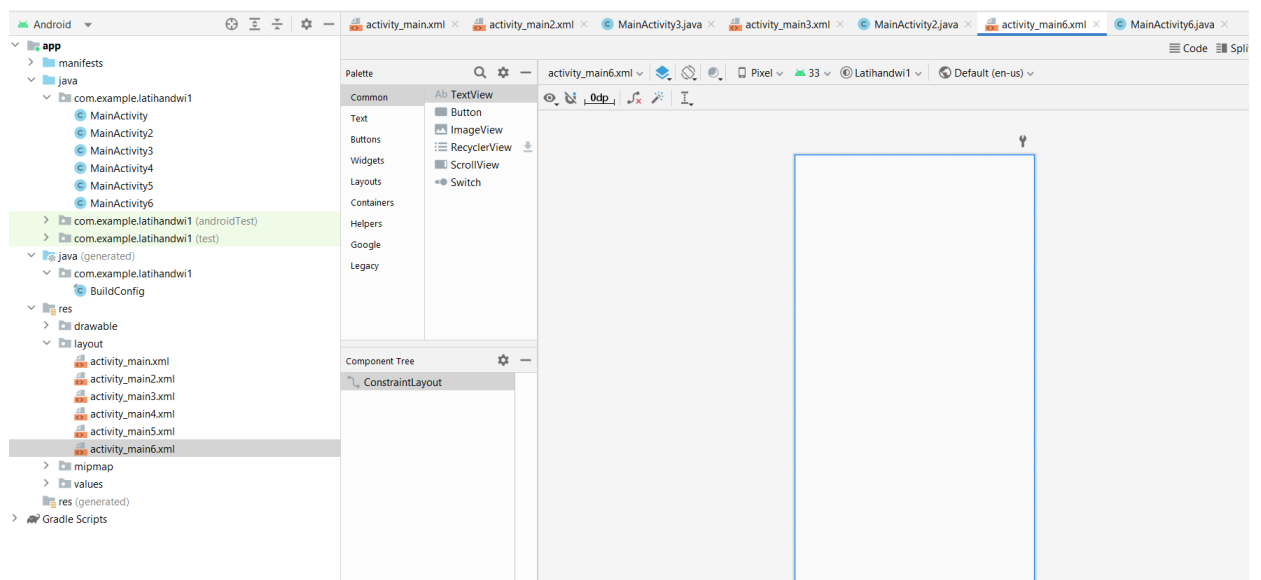


3. Beri nama project seperti berikut ini, lalu klik finish tunggu hingga project

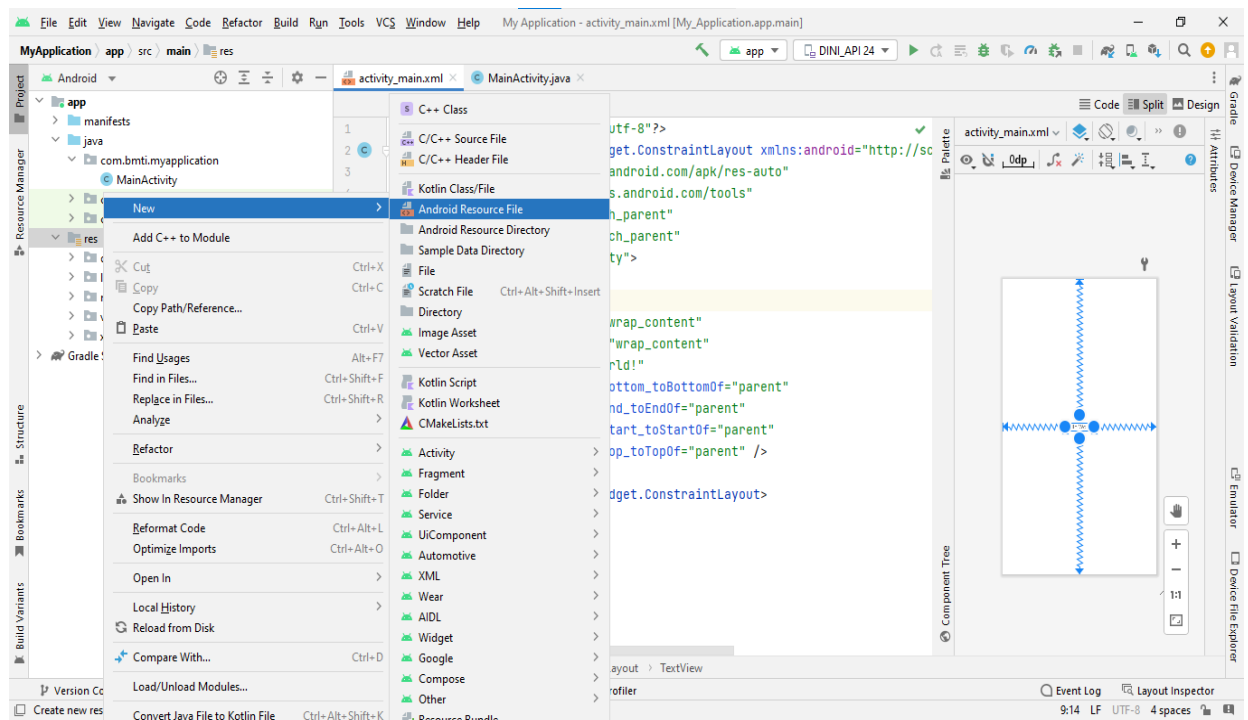
terbentuk.



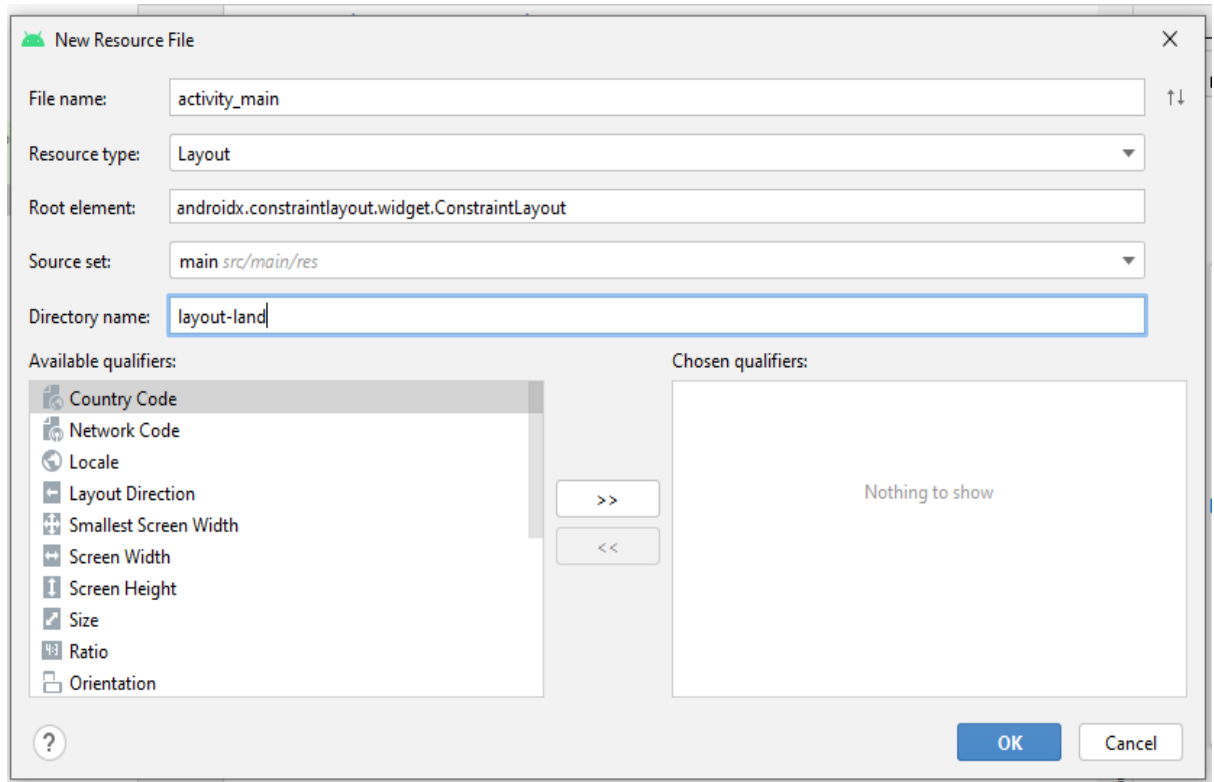
4. Buka "activity_main.xml", kemudian klik tab desain



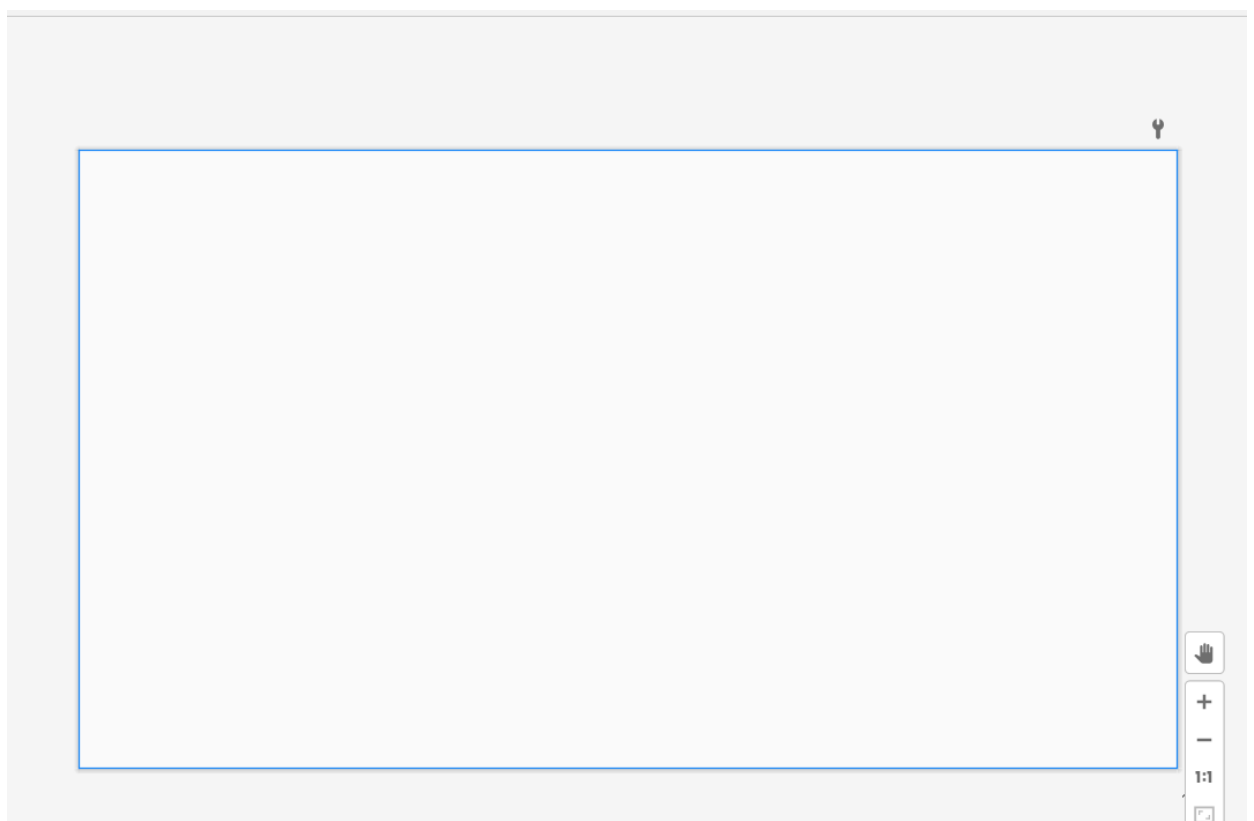
5. Klik kanan folder "res", lalu klik "New" dan kemudian klik "android resource file" untuk membuat layout baru



6. Ketika terlihat tampilan New Resource file maka silakan dibuat “File name” diisi activity_main, lalu di “Resource type” ganti ke “Layout”, dan pada “Directory name” diisi dengan layout-land supaya layarnya menjadi “landscape”, lalu klik “ok”, seperti pada gambar berikut ini.

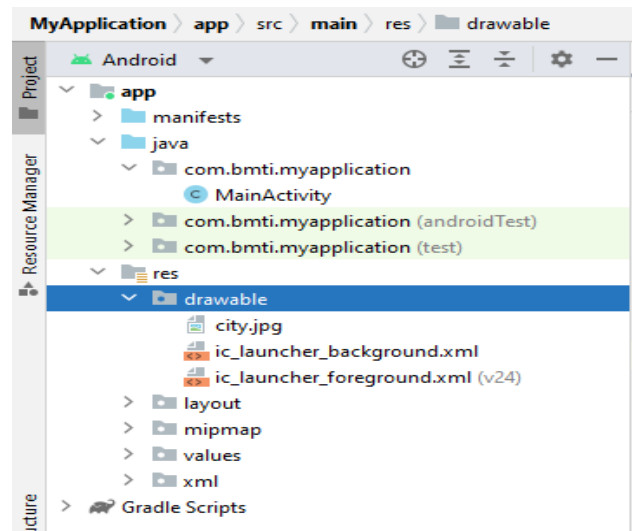


Setelah diklik ok maka tampilan layout akan terlihat landscape seperti berikut ini.



7. Setelah membuat file layout landscape, sekarang coba menambahkan

gambar, dengan cara “copy” gambar yang akan ditampilkan ke folder drawable pada folder res.



8. Buka file “layout/activity_main.xml” lalu ganti kode “<androidx.constraintlayout.widget.ConstraintLayout” menjadi “<ScrollView>” seperti berikut ini.

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity6">

    </ScrollView>
```

Dibawah kode “<ScrollView.>” tambahkan kode “<LinearLayout ” dan klik enter, dan buat kodenya seperti berikut ini.

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```

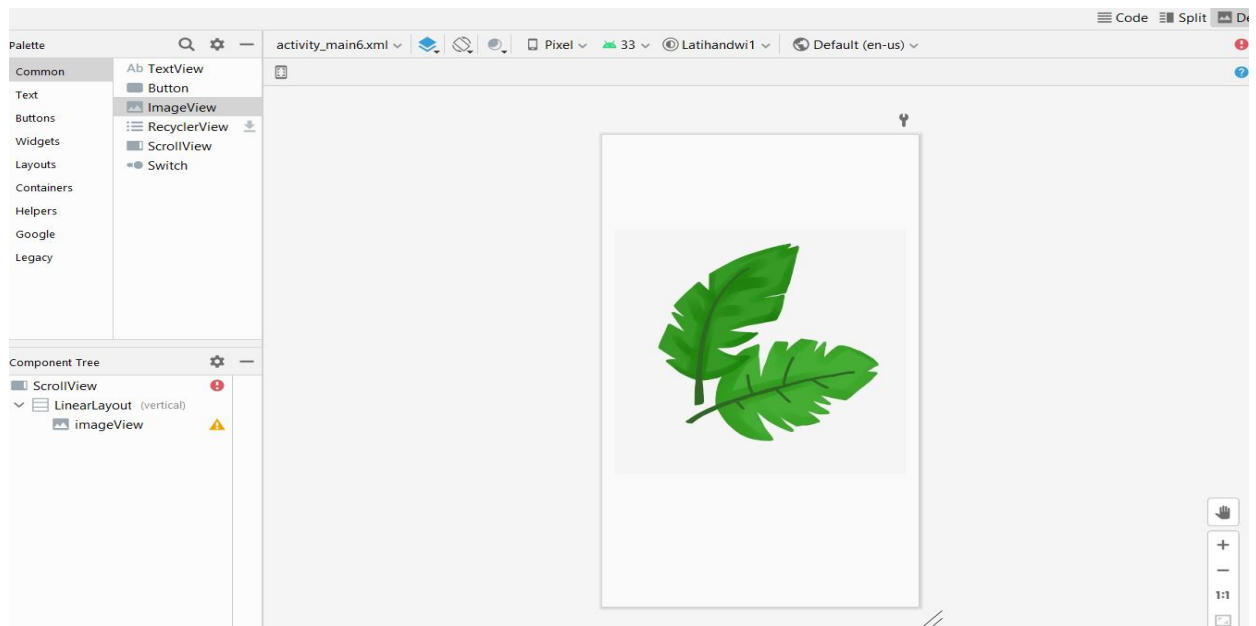
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity6">
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

</LinearLayout>

```

```
</ScrollView>
```

- Setelah itu tambahkan gambar, jika dilakukan melalui desain pilih image view drag ke component tree di bawah linear layout seperti berikut ini.



- Jika menambahkan gambar dilakukan melalui kode, klik tab code lalu buat kode seperti berikut ini.

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity6">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"

```

```
android:orientation="vertical"
android:padding="16dp">
```

```
<ImageView
    android:id="@+id/imageView"
    android:layout_width="match_parent"
    android:layout_height="458dp"
    app:srcCompat="@drawable/ikon1"
    android:scaleType="centerCrop"/>

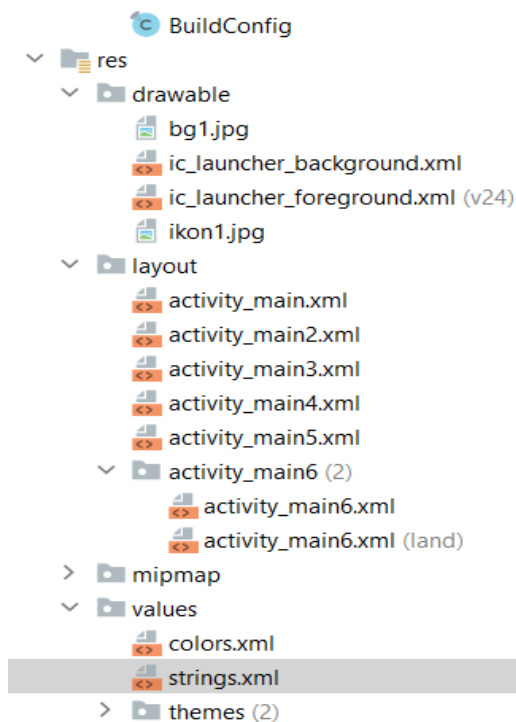
</LinearLayout>
```

```
</ScrollView>
```

11. Selanjutnya tambahkan textview, bisa dari desain atau kode seperti berikut ini.

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="hello world " />
```

12. Dalam folder res, buka folder values, kemudian buka file string.xml, seperti berikut ini.



13. Tambahkan teks pada string.xml seperti berikut ini.

```
<resources>
  <string name="app_name">Layout</string>
  <string name="deskripsi">Hi Gaes selamat datang di BBPPMPV BMTI,
    ini lagi belajar coding responsive layout</string>
</resources>
```

14. Buka file activity_main.xml dan ganti text menjadi deskripsi yang telah dibuat pada file string.xml.

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/deskripsi"/>
```

15. Copy semua kode pada activity_main.xml lalu paste pada file "land/activity_main.xml"

16. Ubah beberapa kode untuk menyesuaikan dengan ukuran seperti berikut ini.

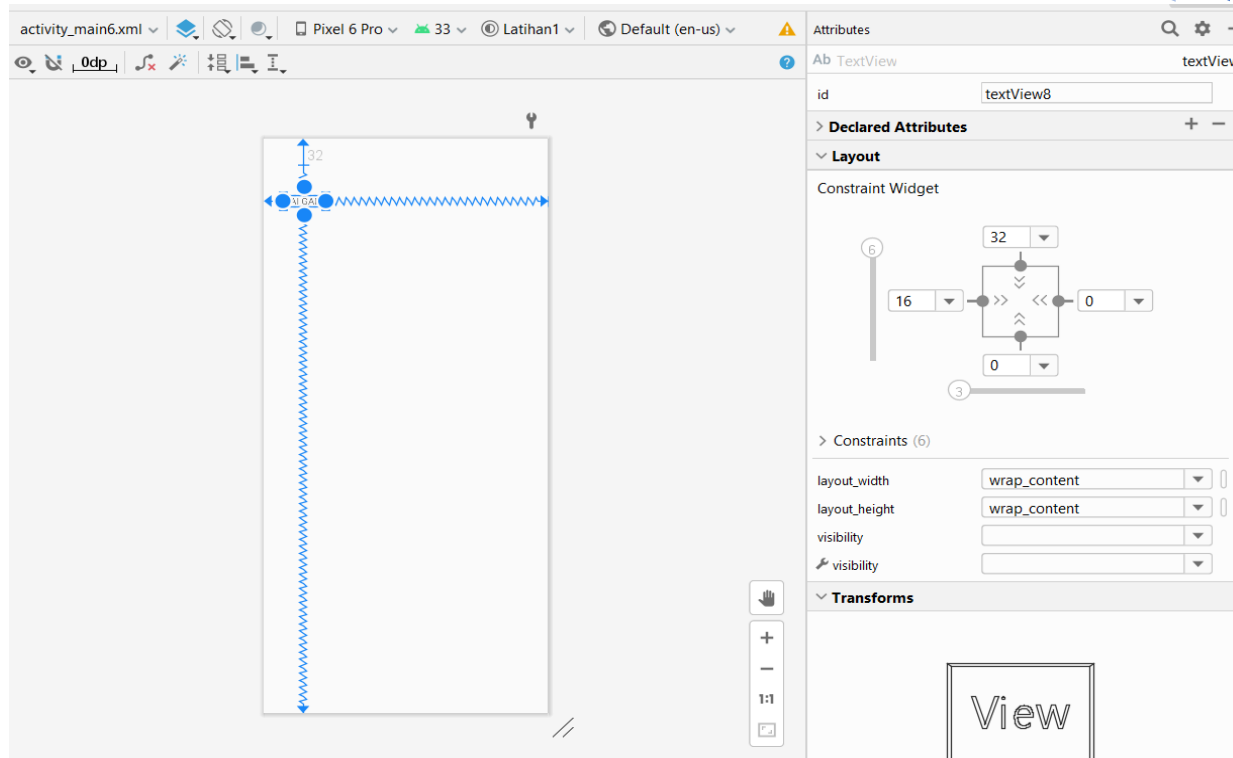
```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="14dp" >
<ImageView
    android:layout_width="250dp"
    android:layout_height="250dp"
    android:layout_marginEnd="14dp"
    android:scaleType="centerCrop"
    android:src="@drawable/city" />
```

17. Jalankan program pada emulator android, maka akan terlihat tampilan dalam bentuk portrait, kemudian untuk melihat tampilan dalam bentuk landscape, silakan rotasikan emulator ke kiri atau ke kanan, sehingga terlihat output program seperti berikut ini.

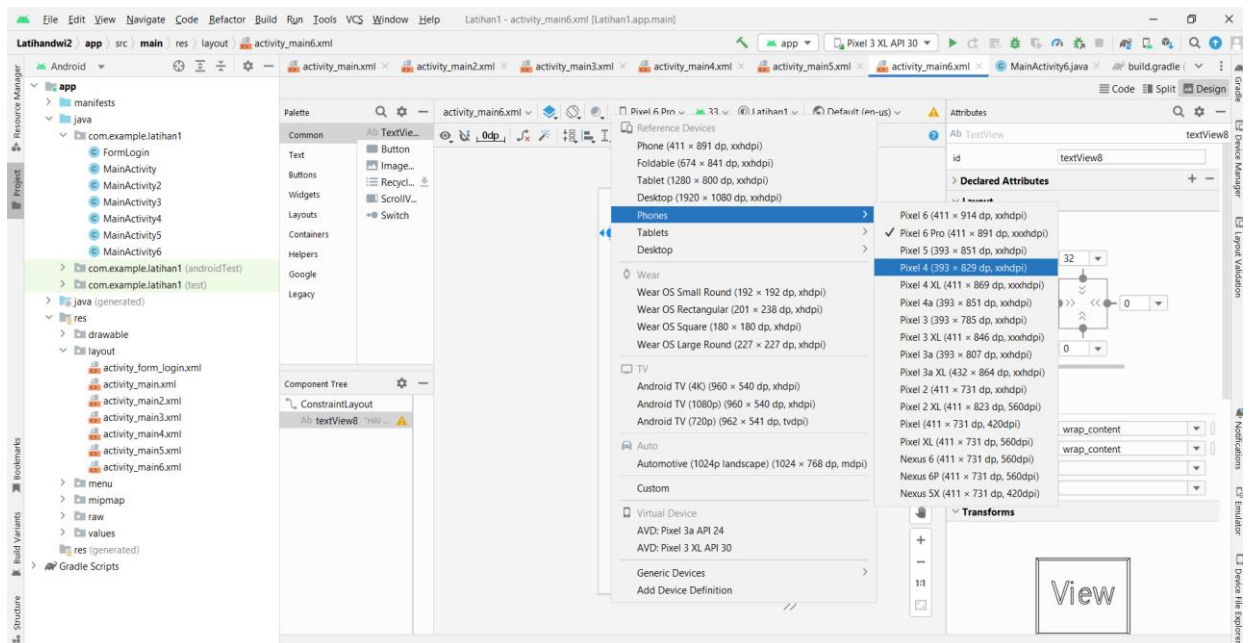


➤ LK 2 Responsive Layout Berdasarkan Ukuran Perangkat

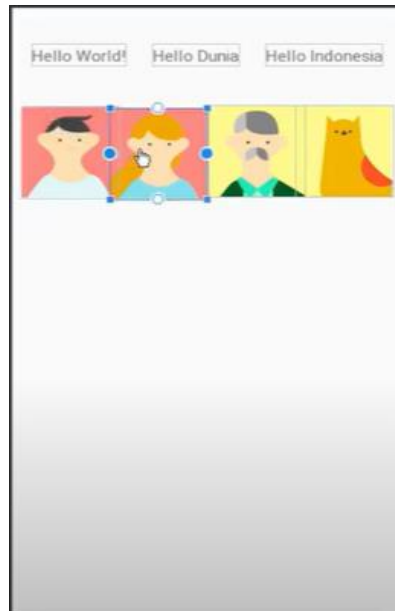
1. Buat project baru, beri nama project, kemudian pilih empty activity, seperti langkah pembuatan project sebelumnya.
2. Buka file `activity_main.xml`, kemudian buka tab desain, tambahkan textview, ganti text dengan Hello, dan atur value constraint layout bagian kiri 16, top 32, seperti berikut ini. Pengaturan ini akan membuat jarak margin kiri sebesar 16 dp dan margin atas 32 dp.



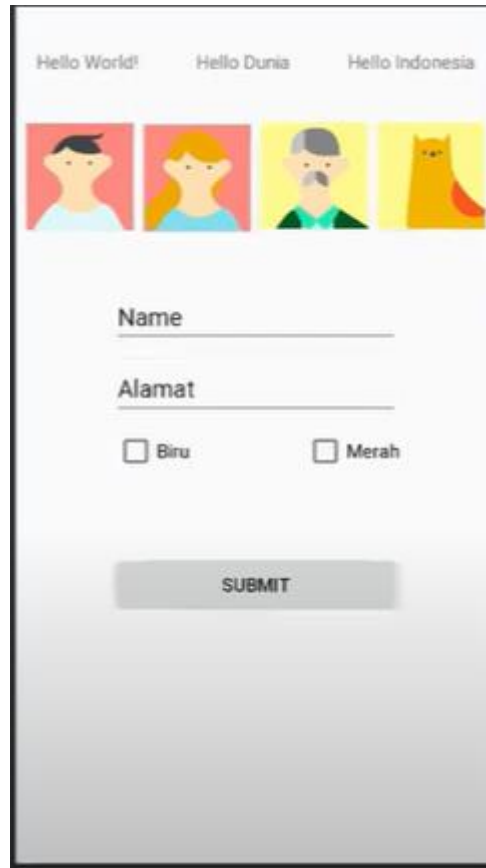
3. Coba pengaturan tersebut pada phone 4 incho, tablet, dan desktop, maka pada semua perangkat akan terlihat jarak margin kiri 16, dan top 32.



4. Tambahkan textview lagi dan image dengan cara pengaturan yang sama dari tab desain saja, sehingga terlihat tampilan seperti berikut ini.



5. Tambahkan edittext dengan tipe plain text, dan button, kemudian lakukan pengaturan value constraint layout, sehingga terlihat tampilan seperti berikut ini.



6. Lakukan pengujian pada berbagai perangkat misalnya pada phone 5 inchi, phone 6 inchi, tablet, serta desktop. Silakan diamati seharusnya layout akan terlihat rapih dan konsisten pada berbagai ukuran perangkat android.

DAFTAR PUSTAKA

Android Developer, Panduan Android Developer,
<https://developer.android.com/guide/> diakses pada 17 Pebruari
2023

Earl Triggs, Android App Development in Android Studio, 2020,
Kindle Edition, USA

Michael Burton, Android Application Development for Dummies 3rd
Edition, 2015, Wiley & Sons Inc, Canada

Neil Smyth, Android 4.0 Development Essentials -Java Edition,
2020, Payload Media Inc, USA